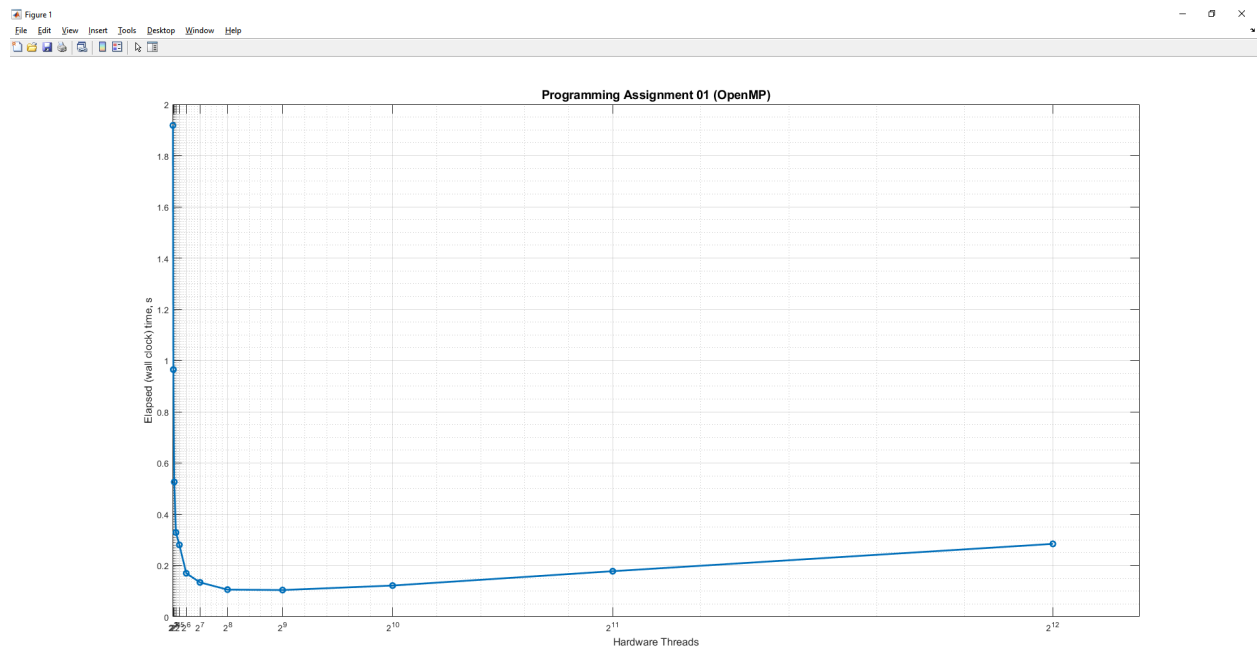


PA1

Runtime Plot



Code Listing

Matrix_parallel.c

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
```

```
#define ORDER 1024
#define AVAL 3.0
#define BVAL 5.0
```

```
int main(int argc, char *argv[]) {
    int Ndim, Pdim, Mdim;
    int i, j, k;
    double tmp;
    double *A, *B, *C;
    Ndim = ORDER;
```

```
Pdim = ORDER;
Mdim = ORDER;
```

```
A = (double*)malloc(Ndim*Pdim*sizeof(double));
B = (double*)malloc(Pdim*Mdim*sizeof(double));
C = (double*)malloc(Ndim*Mdim*sizeof(double));
```

```
double start_time = omp_get_wtime();
```

```
/* initialize matrices */
```

```
for (i = 0; i < Ndim; i++) {
    for (j = 0; j < Pdim; j++)
        *(A+(i*Ndim+j)) = AVAL;
}
```

```
for (i = 0; i < Pdim; i++) {
    for (j = 0; j < Mdim; j++)
        *(B+(i*Ndim+j)) = BVAL;
}
```

```
for (i = 0; i < Ndim; i++) {
    for (j = 0; j < Mdim; j++)
        *(C+(i*Ndim+j)) = 0.0;
}
```

```
unsigned int N_threads = 0;
sscanf(argv[1], "%d", &N_threads);
```

```
omp_set_dynamic(0);
omp_set_num_threads(N_threads);
```

```
/* calculate C matrix */
```

```
#pragma omp parallel for private(tmp, i, j, k)
for (i = 0; i < Ndim; i++) {
    for (j = 0; j < Mdim; j++) {
        tmp = 0.0;
        for (k = 0; k < Pdim; k++) {
            tmp += *(A+(i*Ndim + k)) * *(B+(k*Pdim+j));
        }
        *(C+(i*Ndim+j)) = tmp;
    }
}
```

```

double run_time = omp_get_wtime() - start_time;
double dN = (double)ORDER;
double mflops = 2.0 * dN * dN * dN/(1000000.0 * run_time);

printf("%d, %lf, %lf\n", N_threads, run_time, mflops);

}

```

Matrix_serial.c

```

#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

#define ORDER 1024
#define AVAL 3.0
#define BVAL 5.0

int main(int argc, char *argv[]) {
    int Ndim, Pdim, Mdim;
    int i, j, k;
    double tmp;
    double *A, *B, *C;
    Ndim = ORDER;
    Pdim = ORDER;
    Mdim = ORDER;

    A = (double*)malloc(Ndim*Pdim*sizeof(double));
    B = (double*)malloc(Pdim*Mdim*sizeof(double));
    C = (double*)malloc(Ndim*Mdim*sizeof(double));

    double start_time = omp_get_wtime();

    /* initialize matrices */

    for (i = 0; i < Ndim; i++) {

```

```

    for (j = 0; j < Pdim; j++)
        *(A+(i*Ndim+j)) = AVAL;
}

for (i = 0; i < Pdim; i++) {
    for (j = 0; j < Mdim; j++)
        *(B+(i*Ndim+j)) = BVAL;
}

for (i = 0; i < Ndim; i++) {
    for (j = 0; j < Mdim; j++)
        *(C+(i*Ndim+j)) = 0.0;
}

/* calculate C matrix */
for (i = 0; i < Ndim; i++) {
    for (j = 0; j < Mdim; j++) {
        tmp = 0.0;
        for (k = 0; k < Pdim; k++) {
            tmp += *(A+(i*Ndim + k)) * *(B+(k*Pdim+j));
        }
        *(C+(i*Ndim+j)) = tmp;
    }
}

double run_time = omp_get_wtime() - start_time;
double dN = (double)ORDER;
double mflops = 2.0 * dN * dN * dN/(1000000.0 * run_time);

printf("%lf runtime, %lf mflops\n", run_time, mflops);

}

```

Program output

```
rsotto@dgx:~$ ./matrix 1
1, 3.855172, 557.039653
rsotto@dgx:~$ ./matrix 2
2, 1.930078, 1112.641109
rsotto@dgx:~$ ./matrix 4
4, 0.973363, 2206.251510
rsotto@dgx:~$ ./matrix 8
8, 0.511267, 4200.316309
rsotto@dgx:~$ ./matrix 16
16, 0.407078, 5275.364423
rsotto@dgx:~$ ./matrix 32
32, 0.274686, 7817.965560
rsotto@dgx:~$ ./matrix 64
64, 0.180913, 11870.232844
rsotto@dgx:~$ ./matrix 128
128, 0.140448, 15290.293929
rsotto@dgx:~$ ./matrix 256
256, 0.136335, 15751.554979
rsotto@dgx:~$ ./matrix 512
512, 0.109703, 19575.484562
rsotto@dgx:~$ ./matrix 1024
1024, 0.139023, 15446.946118
rsotto@dgx:~$ ./matrix 2048
2048, 0.193625, 11090.927982
rsotto@dgx:~$ ./matrix 4096
4096, 0.291594, 7364.629461
rsotto@dgx:~$ ./matrix_serial
3.828061 runtime, 560.984747 mflops
rsotto@dgx:~$ █
```

MATLAB plot script

```
close all
clc
data = load('data.txt', '-ascii');
threads = data(:,1);
times = data(:,2);
figure;
plot(threads, times, 'o-', 'LineWidth', 2);
set(gca, 'XTick', threads);
set(gca, 'XTickLabel', arrayfun(@(x) sprintf('2^{%d}', log2(x)), threads,
'UniformOutput', false));
xlabel('Hardware Threads', 'FontSize', 12);
ylabel('Elapsed (wall clock) time, s', 'FontSize', 12);
title('Programming Assignment 01 (OpenMP)', 'FontSize', 14);
set(gcf, 'Color', 'white');
grid on;
ax = gca;
ax.YMinorGrid = 'on';
```

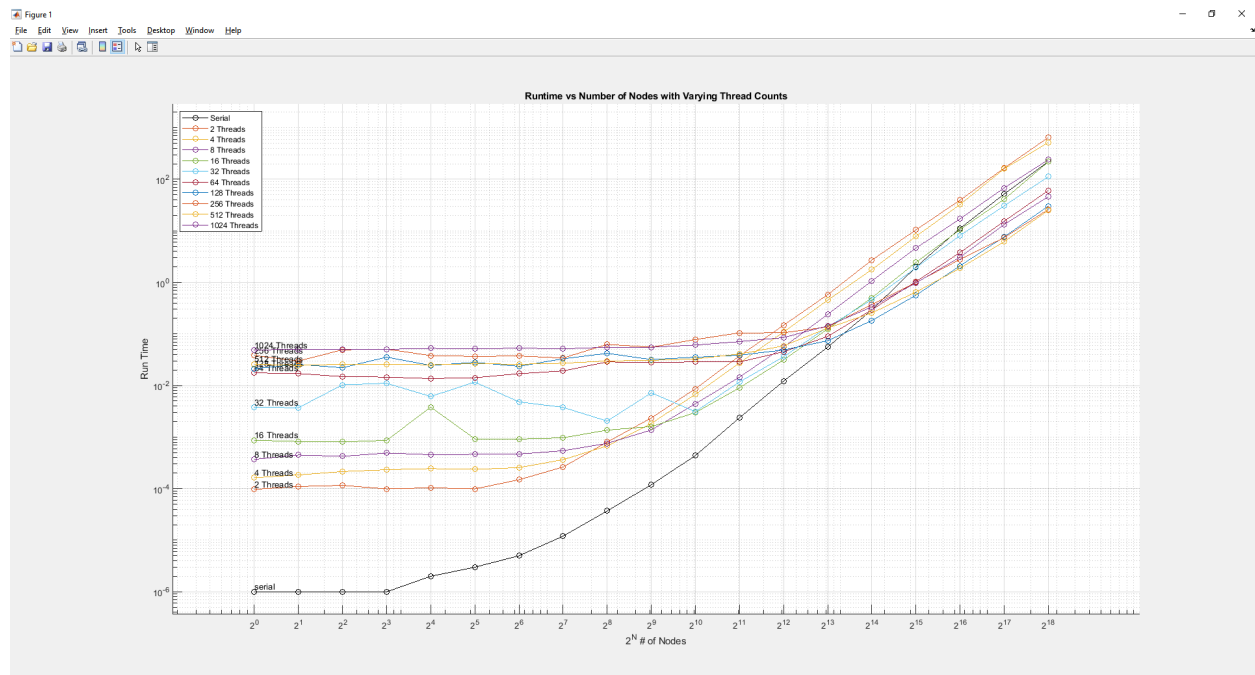
```
ax.XMinorGrid = 'on';  
figure(1);
```

Discussion of the optimal thread count value

It appears that the optimal thread count value I found was 512 threads given an order of 1024, although there was a minor amount of variance in the run times through repeated trials. A thread count of 256 had similar results, but just barely lost out to 512 threads. There was significant speed up in the parallel implementation compared to the serial implementation given more threads, which was to be expected. Running the parallel implementation with one thread resulted in a similar, but slightly slower run time than the serial implementation, likely due to the overhead of OpenMP threads. Going above 512 threads resulted in increased runtimes, so it is possible that the true optimal amount of threads would lie between 256 and 512.

PA2

Runtime Plot



Comments

It is notable that for most values of n , it is significantly worse than the serial implementation for values of N smaller than 2^{13} . This is likely due to the overhead cost of multithreading, as well as collisions caused by the locking mechanisms used for parallel synchronization. It is only once N is greater than 2^{13} that multithreading results in smaller runtime or a “break-even” point is reached, but noticeably not for every multithreading configuration. Past that point, 2, 4, and 8 threads are still worse for runtime than the serial implementation, but thread counts up to 1024 manage to surpass the serial implementation.

The runtime is shown logarithmically, so the speedup for higher thread counts at large node counts is significant – thread count of 512 displays a speedup of around 4x. For parallelizing insertions into a linked list, it is beneficial if the amount of node insertions is large (around 2^{14}), and the amount of threads is larger than 8.

Parallel C Code – bl_insert.c

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct Node{
    int value;
    struct Node* next;
    struct Node* prev;
    omp_lock_t lock;
} typedef Node;

int main(int argc, char* argv[]) {

    double start_time = omp_get_wtime();

    // read arguments
    int nThreads = atoi(argv[1]);
    int a = atoi(argv[2]);

    // omp settings
    omp_set_dynamic(0);
    omp_set_num_threads(nThreads);

    // pointers
    Node* head = NULL;
    Node* tail = NULL;
    Node* p, *prev, *newNode = NULL;
    int value = 0;

    // create initial head/tail node
    head = (Node*)calloc(1, sizeof(Node));
    head->value = 0;
    head->next = NULL;
    head->prev = NULL;
    omp_init_lock(&head->lock);

    unsigned long long N = 1 << a;
    unsigned long long k = 0;

    #pragma omp parallel for private(k, value, newNode, p, prev)
```

```

for (k = 0; k < N; k++) {
    prev = head;
    value = rand() % 1000 + 1; // random value

    omp_set_lock(&prev->lock); // lock during traversal
    p = head->next;

    if (p != NULL) omp_set_lock(&p->lock);
    // iterate through linked list to find where to insert node
    while (p != NULL) {
        if (p->value >= value) break;
        omp_unset_lock(&prev->lock); // unlock previous node

        prev = p;
        p = p->next;
        if (p != NULL) omp_set_lock(&p->lock); // previous p holds lock, p holds lock
    }
    // p holds the place where the node should be inserted before

    // create new node
    newNode = (Node*)calloc(1, sizeof(Node));
    newNode->value = value;
    newNode->next = p;
    newNode->prev = prev;
    if (p != NULL) {
        p->prev = newNode;
    }
    newNode->prev->next = newNode;
    omp_init_lock(&newNode->lock);

    // unlock both held locks
    if (p != NULL) omp_unset_lock(&p->lock);
    omp_unset_lock(&prev->lock);
}

// checking forward and backward traversal
// p = head->next;
// while (p != NULL) {
//     printf("%d\n", p->value);
//     if (p->next == NULL) {
//         tail = p;
//     }
//     p = p->next;
// }

```

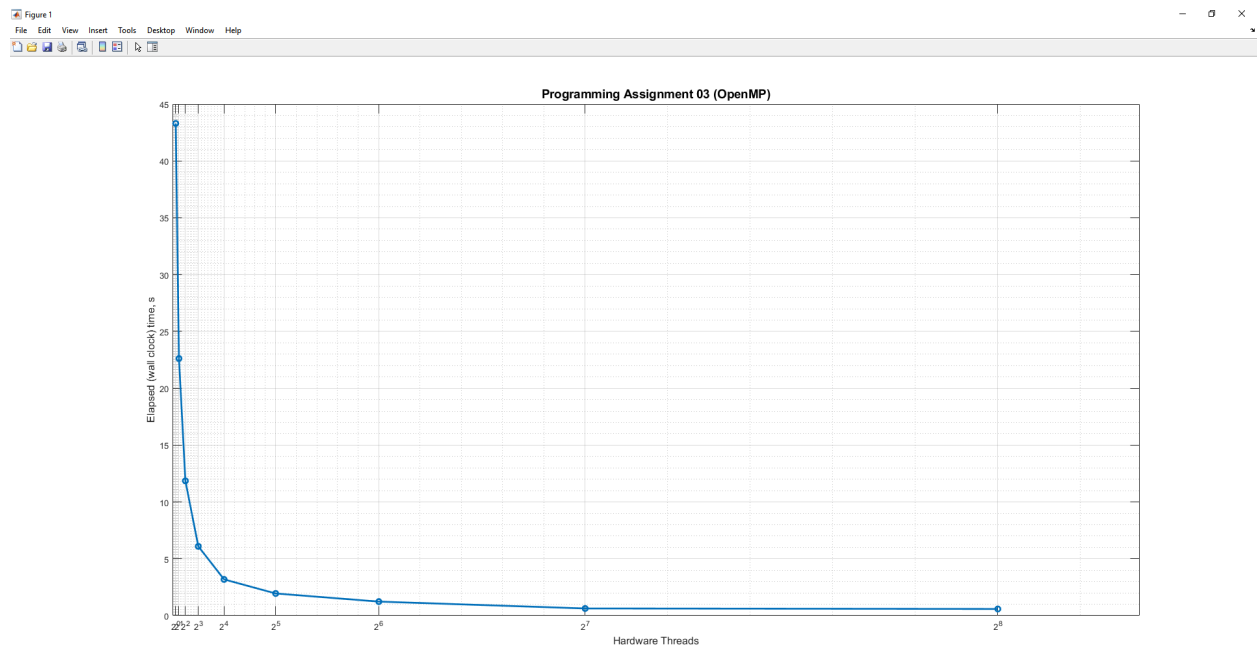
```
// p = tail;
// while (p != NULL) {
//     printf("%d\n", p->value);
//     p = p->prev;
// }
double run_time = omp_get_wtime() - start_time;

printf("%d, %llu, %lf\n", nThreads, N, run_time);

return 0;
}
```

PA3

Runtime Plot



Comments

It appears that the optimal thread count value I found was 256 threads. There was significant speed up in the parallel implementation given more threads, which was to be expected. The error was very minor (in the order of 10^{-09}) and did not change drastically through using more threads, though it could be noted that the error was smallest when only using 1 thread.

Data (NumThreads, Approx, Exact, |Approx-Exact|, Runtime)

1	2.0561675786	2.0561675836	4.9346553510e-09	43.296352
2	2.0561675786	2.0561675836	4.9348951592e-09	22.619585
4	2.0561675786	2.0561675836	4.9348498621e-09	11.855170
8	2.0561675786	2.0561675836	4.9347352871e-09	6.095932
16	2.0561675786	2.0561675836	4.9348365394e-09	3.186908
32	2.0561675786	2.0561675836	4.9348392039e-09	1.948292
64	2.0561675786	2.0561675836	4.9348227726e-09	1.233952
128	2.0561675786	2.0561675836	4.9348152231e-09	0.629846
256	2.0561675786	2.0561675836	4.9347974596e-09	0.584878

C Code – quad.c

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// function
double f(double x) {
    double c = cos(x);
    return acos(c / (1.0 + 2.0 * c));
}

int main(int argc, char* argv[]) {
    double start_time = omp_get_wtime();
    // exact answer to compare approximation to
    double exact = 5 * (M_PI * M_PI) / 24;
    // read arguments
    int Nthrds = atoi(argv[1]);
    // integral bounds
    double a = 0.0, b = M_PI / 2;

    // omp settings
    omp_set_dynamic(0);
    omp_set_num_threads(Nthrds);

    long long n = 1000000000LL;
    long long N = n / 2;
    const double h = (b - a) / n;

    double approx = 0.0;
    // 1/3 Simpson's Rule
    #pragma omp parallel for reduction(+:approx)
    for (long long j = 1; j < N; ++j) {
        approx += f(a + (2*j-2)*h)
            + 4*f(a + (2*j-1)*h)
            + f(a + (2*j)*h);
    }
    approx *= h/3;

    double run_time = omp_get_wtime() - start_time;
    // error calculation and runtime
    printf("%d %.10f %.10f %.10e %fn", Nthrds, approx, exact, fabs(exact - approx), run_time);
```

```
    return 0;
}
```

Makefile

```
all: quad
```

```
quad: quad.o
    gcc -fopenmp quad.c -o quad -lm
```

```
THREADS = 1 2 4 8 16 32 64 128 256
OUTPUT_FILE = data.txt
```

```
run: quad
    @for thread in $(THREADS); do \
        ./quad $$thread >> $(OUTPUT_FILE); \
    done \
```

MATLAB plot script

```
close all
clc
data = load('data.txt', '-ascii');
threads = data(:,1);
runtime = data(:,5);
error = data(:,3);
figure;
plot(threads, runtime, 'o-', 'LineWidth', 2);
set(gca, 'XTick', threads);
set(gca, 'XTickLabel', arrayfun(@(x) sprintf('2^{%d}', log2(x)), threads,
'UniformOutput', false));
xlabel('Hardware Threads', 'FontSize', 12);
ylabel('Elapsed (wall clock) time, s', 'FontSize', 12);
title('Programming Assignment 03 (OpenMP)', 'FontSize', 14);
set(gcf, 'Color', 'white');
grid on;
ax = gca;
ax.YMinorGrid = 'on';
ax.XMinorGrid = 'on';
figure(1);
```


PA4

Comments

It appears that an array requires 2^{24} elements to see a runtime improvement of GPU device accelerated reduction over CPU sequential summation. The speedup for $N = 23$ (calculated as $\text{time_CPU}/\text{time_GPU}$) was slightly greater than 1.0, being 1.050271. After this point, the speedup got significantly better with each N past 24 ($N = 25 \rightarrow 2.446656$, $N = 26 \rightarrow 4.750253$, $N = 27 \rightarrow 11.686365$, $N = 28 \rightarrow 15.465755$).

The error of the CPU sum vs the GPU sum also increased (not directly measured) with higher N values, likely due to a combination of floating point error and limitations of the floating point type (using floats instead of doubles).

CUDA Code – reduction.cu

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define cudaCheckError() { \
    cudaError_t e=cudaGetLastError(); \
    if(e!=cudaSuccess) { \
        printf("Cuda failure %s:%d: '%s'\n",__FILE__,__LINE__,cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
    } \
} \

#define BLOCK_SIZE 512 // You can change this

// device
__global__ void reduction(float *input, float *output, int len) {
    // Load a segment of the input vector into shared memory
    __shared__ float partialSum[2 * BLOCK_SIZE];
    unsigned int t = threadIdx.x, start = 2 * blockIdx.x * BLOCK_SIZE;
    if (start + t < len)
        partialSum[t] = input[start + t];
    else
        partialSum[t] = 0;
    if (start + BLOCK_SIZE + t < len)
        partialSum[BLOCK_SIZE + t] = input[start + BLOCK_SIZE + t];
    else
        partialSum[BLOCK_SIZE + t] = 0;
    // Traverse the reduction tree
```

```

/*
strides will assume values:
512
256
128
64
32
16
8
4
2
1
*/
for (unsigned int stride = BLOCK_SIZE; stride >= 1; stride >>= 1) {
    __syncthreads();
    if (t < stride)
        partialSum[t] += partialSum[t + stride];
}
// Write the computed sum of the block to the output vector at the
// correct index
if (t == 0)
    output[blockIdx.x] = partialSum[0];
}

// host
int main(int argc, char* argv[]) {

    // READ INPUTS
    unsigned int N = atoi(argv[1]);
    printf("N = %d\n", N);
    // 2^N elements
    unsigned int numInputElements = (1 << N);

    // allocate memory on host
    float *hostInput = (float*)calloc(numInputElements, sizeof(float));
    // initialize input array
    for (unsigned int i = 1; i < numInputElements; ++i) {
        hostInput[i] = (float)i;
    }

    // PERFORM SEQUENTIAL SUM ON HOST
    clock_t t;

```

```

t = clock();
// host computation
float hostOutput = 0.0f;
for (unsigned int i = 0; i < numInputElements; ++i) {
    hostOutput += hostInput[i];
}

t = clock() - t;
float cpuElapsedTime = ((float)t/CLOCKS_PER_SEC * 1000.0);
printf("host result: %f\n", hostOutput);
printf("host elapsed time: %f ms\n", cpuElapsedTime);

// PERFORM SEQUENTIAL SUM ON DEVICE
float *deviceInput, *deviceOutput;

// allocate memory on device
unsigned int numBlocks = (numInputElements + (2*BLOCK_SIZE - 1)) / (2*BLOCK_SIZE);
cudaMalloc((void**)&deviceInput, numInputElements * sizeof(float));
cudaMalloc((void**)&deviceOutput, numBlocks * sizeof(float));
// copy host inputs to device
cudaMemcpy(deviceInput, hostInput, numInputElements * sizeof(float),
cudaMemcpyHostToDevice);

cudaEvent_t start, stop;
float gpuElapsedTime;

cudaEventCreate(&start);
cudaEventRecord(start,0);

// invoke reduction kernel
dim3 dimGrid(numBlocks, 1, 1);
dim3 dimBlock(BLOCK_SIZE, 1, 1);
reduction<<<dimGrid, dimBlock>>>(deviceInput, deviceOutput, numInputElements);

cudaCheckError();

cudaEventCreate(&stop);
cudaEventRecord(stop,0);
cudaEventSynchronize(stop);

// copy from device back to host
float *hostPartial = (float*)malloc(numBlocks * sizeof(float));

```

```
    cudaMemcpy(hostPartial, deviceOutput, numBlocks * sizeof(float),
cudaMemcpyDeviceToHost);
```

```
    float gpuOutput = 0.0f;
    for (unsigned int i = 0; i < numBlocks; i++) {
        gpuOutput += hostPartial[i];
    }
```

```
    cudaEventElapsedTime(&gpuElapsedTime, start, stop);
```

```
    printf("device result: %f\n", gpuOutput);
    printf("device elapsed time: %f ms\n", gpuElapsedTime);
```

```
    // report result as speedup (sequential/parallel)
    float speedup = cpuElapsedTime/gpuElapsedTime;
    printf("Speedup = %f\n", speedup);
```

```
    // free allocated memory
    cudaFree(deviceInput);
    cudaFree(deviceOutput);
    free(hostInput);
    free(hostPartial);
```

```
    return 0;
}
```

Makefile

all: reduction

reduction: reduction.cu
 nvcc reduction.cu -o reduction

N = 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
OUTPUT_FILE = data.txt

run: reduction
 @rm -f \$(OUTPUT_FILE)
 @for N in \$(N); do \
 ./reduction \$\$N >> \$(OUTPUT_FILE); \
 done \

Runtime Transcript

N = 1

host result: 1.000000

host elapsed time: 0.000000 ms

device result: 1.000000

device elapsed time: 25.613312 ms

Speedup = 0.000000

N = 2

host result: 6.000000

host elapsed time: 0.001000 ms

device result: 6.000000

device elapsed time: 47.591423 ms

Speedup = 0.000021

N = 3

host result: 28.000000

host elapsed time: 0.001000 ms

device result: 28.000000

device elapsed time: 72.538109 ms

Speedup = 0.000014

N = 4

host result: 120.000000

host elapsed time: 0.001000 ms

device result: 120.000000

device elapsed time: 56.162304 ms

Speedup = 0.000018

N = 5

host result: 496.000000

host elapsed time: 0.001000 ms

device result: 496.000000

device elapsed time: 46.475266 ms

Speedup = 0.000022

N = 6

host result: 2016.000000

host elapsed time: 0.001000 ms

device result: 2016.000000

device elapsed time: 31.889408 ms

Speedup = 0.000031

N = 7

host result: 8128.000000

host elapsed time: 0.000000 ms

device result: 8128.000000

device elapsed time: 34.079742 ms

Speedup = 0.000000

N = 8

host result: 32640.000000

host elapsed time: 0.001000 ms

device result: 32640.000000

device elapsed time: 49.423359 ms

Speedup = 0.000020

N = 9

host result: 130816.000000

host elapsed time: 0.003000 ms

device result: 130816.000000

device elapsed time: 32.291840 ms

Speedup = 0.000093

N = 10

host result: 523776.000000

host elapsed time: 0.004000 ms

device result: 523776.000000

device elapsed time: 36.471809 ms

Speedup = 0.000110

N = 11

host result: 2096128.000000

host elapsed time: 0.008000 ms

device result: 2096128.000000

device elapsed time: 33.332226 ms

Speedup = 0.000240

N = 12

host result: 8386560.000000

host elapsed time: 0.014000 ms

device result: 8386560.000000

device elapsed time: 58.628098 ms

Speedup = 0.000239

N = 13

host result: 33549136.000000

host elapsed time: 0.028000 ms

device result: 33550336.000000

device elapsed time: 67.066879 ms

Speedup = 0.000417

N = 14

host result: 134204240.000000

host elapsed time: 0.054000 ms

device result: 134209536.000000

device elapsed time: 30.551041 ms

Speedup = 0.001768

N = 15

host result: 536841088.000000

host elapsed time: 0.108000 ms
device result: 536854528.000000
device elapsed time: 38.991520 ms
Speedup = 0.002770
N = 16
host result: 2147421184.000000
host elapsed time: 0.215000 ms
device result: 2147450880.000000
device elapsed time: 43.314014 ms
Speedup = 0.004964
N = 17
host result: 8589806592.000000
host elapsed time: 0.430000 ms
device result: 8589869056.000000
device elapsed time: 39.423519 ms
Speedup = 0.010907
N = 18
host result: 34359984128.000000
host elapsed time: 0.874000 ms
device result: 34359672832.000000
device elapsed time: 61.869888 ms
Speedup = 0.014126
N = 19
host result: 137439117312.000000
host elapsed time: 1.719000 ms
device result: 137438887936.000000
device elapsed time: 46.612545 ms
Speedup = 0.036878
N = 20
host result: 549821087744.000000
host elapsed time: 3.452000 ms
device result: 549755748352.000000
device elapsed time: 48.983871 ms
Speedup = 0.070472
N = 21
host result: 2199644798976.000000
host elapsed time: 6.892000 ms
device result: 2199023124480.000000
device elapsed time: 30.299904 ms
Speedup = 0.227459
N = 22
host result: 8809264185344.000000
host elapsed time: 13.775001 ms
device result: 8796092497920.000000

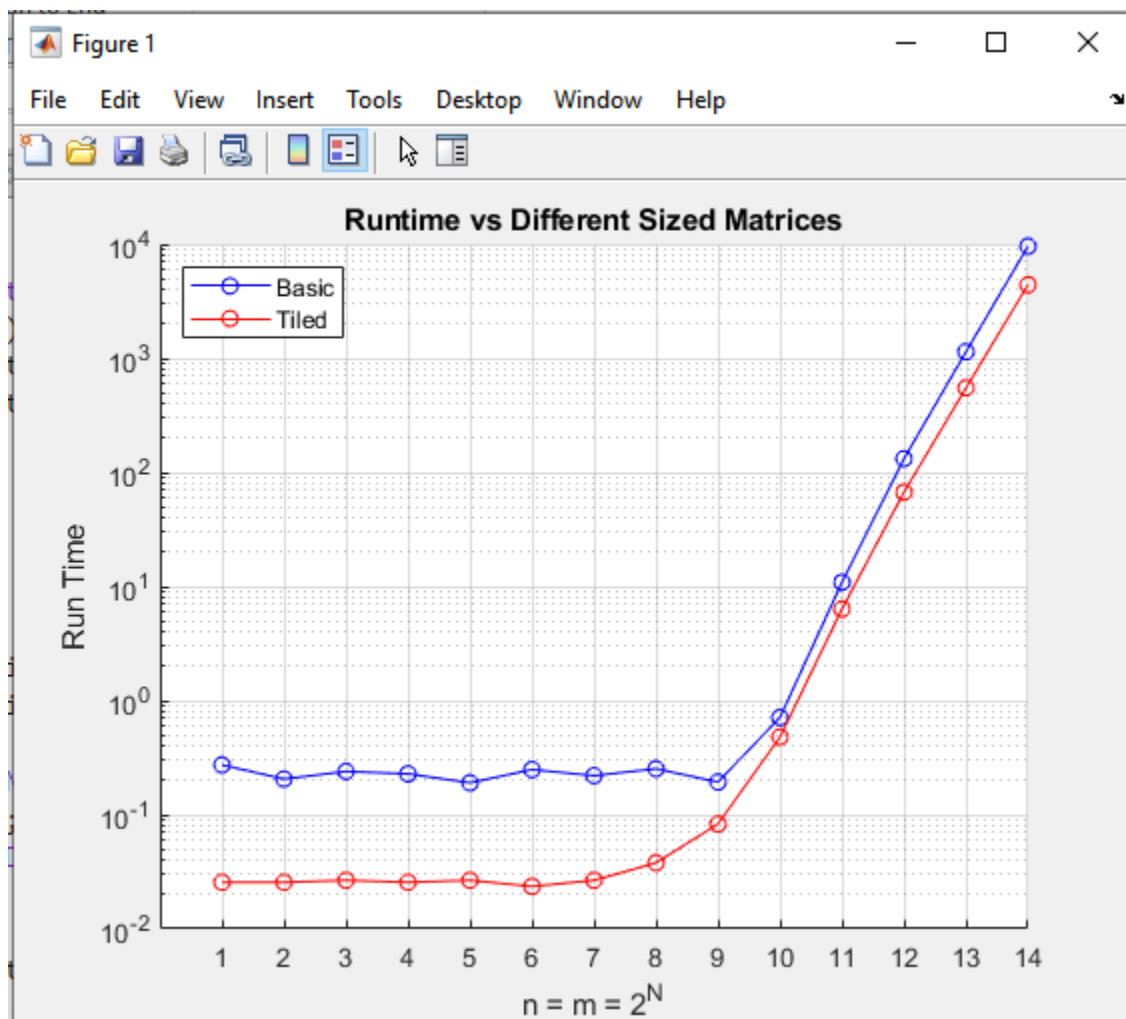
device elapsed time: 29.591007 ms
Speedup = 0.465513
N = 23
host result: 35197542203392.000000
host elapsed time: 27.577999 ms
device result: 35183482896384.000000
device elapsed time: 39.720352 ms
Speedup = 0.694304
N = 24
host result: 146610319261696.000000
host elapsed time: 55.459999 ms
device result: 140736599162880.000000
device elapsed time: 52.805408 ms
Speedup = 1.050271
N = 25
host result: 562949953421312.000000
host elapsed time: 110.443001 ms
device result: 562949081006080.000000
device elapsed time: 45.140385 ms
Speedup = 2.446656
N = 26
host result: 1125899906842624.000000
host elapsed time: 221.456009 ms
device result: 2251799008378880.000000
device elapsed time: 46.619839 ms
Speedup = 4.750253
N = 27
host result: 2251799813685248.000000
host elapsed time: 441.911011 ms
device result: 9007198717870080.000000
device elapsed time: 37.814240 ms
Speedup = 11.686365
N = 28
host result: 4503599627370496.000000
host elapsed time: 886.458984 ms
device result: 36029325299941376.000000
device elapsed time: 57.317535 ms
Speedup = 15.465755

PA5

Comments

It appears that the tiled implementation (TILE_WIDTH = 16) manages to outperform the basic implementation for all values of N tested. It is expected for shared memory tiles to have better performance for large N values, but not necessarily for small N. I was having issues with the basic kernel taking significantly longer (~60ms for N = 1-9) that I assumed was due to the drivers warming up on first kernel invocation, so I included a dummy kernel which made the results closer as shown below.

Another cause could be the kernel dimensions, since I attempted to use different kernel dimensions as well between the basic and tiled implementations. Overall, while the large N results are in line with expectations, the small N results are unexpected.



CUDA Code - mmult.cu

```
#include <stdio.h>
#include <stdlib.h>

#define cudaCheckError() { \
    cudaError_t e=cudaGetLastError(); \
    if(e!=cudaSuccess) { \
        printf("Cuda failure %s:%d: '%s'\n",__FILE__,__LINE__,cudaGetErrorString(e)); \
        exit(EXIT_FAILURE); \
    } \
}

#define TILE_WIDTH 16

// basic matrix multiply kernel
__global__ void matrixMultiply(float* M, float* N, float* P, int Width) {
    int Row = blockIdx.y * blockDim.y + threadIdx.y;
    int Col = blockIdx.x * blockDim.x + threadIdx.x;
    float Pvalue = 0;

    // Participating threads
    if (Row < Width && Col < Width) {
        for (int i = 0; i < Width; i++) {
            Pvalue += M[Row*Width + i] * N[i*Width + Col];
        }
        P[Row*Width+Col] = Pvalue;
    }
}

// tiled matrix multiplication kernel
__global__ void matrixTiledMultiply(float* M, float* N, float* P, int Width) {
    __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x; int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    int Row = by * blockDim.y + ty;
    int Col = bx * blockDim.x + tx;
    float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    for (int p = 0; p < (Width + TILE_WIDTH - 1) / TILE_WIDTH; p++) {
```

```

// Collaborative loading of M and N tiles into shared memory
if (Row < Width && p*TILE_WIDTH + tx < Width) {
    ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
}
else {
    ds_M[ty][tx] = 0;
}
if (Col < Width && p*TILE_WIDTH + ty < Width) {
    ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
}
else {
    ds_N[ty][tx] = 0;
}

__syncthreads();
for (int i = 0; i < TILE_WIDTH; ++i) {
    Pvalue += ds_M[ty][i] * ds_N[i][tx];
}
__syncthreads();
}
if (Row < Width && Col < Width) {
    P[Row*Width+Col] = Pvalue;
}
}

```

```

int main(int argc, char* argv[]) {
    // read inputs
    unsigned int pow2 = atoi(argv[1]);
    printf("N = %d\n", pow2);
    unsigned int dim = (1 << pow2);

    // matrix dimensions
    unsigned int m, p, n;
    m = dim;
    p = dim;
    n = dim;

    // allocate host memory for A, B, C arrays
    float* hA = (float*)calloc(m*p, sizeof(float));
    float* hB = (float*)calloc(p*n, sizeof(float));
    float* hC = (float*)calloc(m*n, sizeof(float));

    // fill A and B with random values

```

```

srand(0);
for (unsigned int i = 0; i < m*p; i++) {
    hA[i] = rand() / (float)RAND_MAX;
}
for (unsigned int i = 0; i < p*n; i++) {
    hB[i] = rand() / (float)RAND_MAX;
}

// DEVICE START //

// initialize and allocate device memory
float* dA, *dB, *dC;
cudaMalloc(&dA, sizeof(float) * m * p);
cudaMalloc(&dB, sizeof(float) * p * n);
cudaMalloc(&dC, sizeof(float) * m * n);
// copy host memory to device
cudaMemcpy(dA, hA, sizeof(float) * m * p, cudaMemcpyHostToDevice);
cudaMemcpy(dB, hB, sizeof(float) * p * n, cudaMemcpyHostToDevice);
// initialize thread block and kernel grid dimensions
dim3 dimBlock_basic(16,16);
dim3 dimGrid_basic((dim + 15)/16, (dim + 15)/16);
dim3 dimBlock_tiled(TILE_WIDTH, TILE_WIDTH);
dim3 dimGrid_tiled((dim + TILE_WIDTH - 1)/TILE_WIDTH, (dim + TILE_WIDTH -
1)/TILE_WIDTH);
// time measurements (measure only the kernel invocations)
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
// warm up drivers
matrixMultiply<<<1,1>>>(dA,dB,dC,1);
cudaDeviceSynchronize();
// invoke CUDA kernel
cudaFree(0);
cudaEventRecord(start);
matrixMultiply<<<dimGrid_basic, dimBlock_basic>>>(dA, dB, dC, dim);
cudaCheckError();
cudaEventRecord(stop);
cudaEventSynchronize(stop);

float ms_basic = 0;
cudaEventElapsedTime(&ms_basic, start, stop);

cudaMemset(dC, 0, sizeof(float) * m * n);

```

```

// invoke CUDA kernel
cudaEventRecord(start);
matrixTiledMultiply<<<dimGrid_tiled, dimBlock_tiled>>>(dA, dB, dC, dim);
cudaCheckError();
cudaEventRecord(stop);
cudaEventSynchronize(stop);

float ms_tiled = 0;
cudaEventElapsedTime(&ms_tiled, start, stop);
// copy results from device to host
cudaDeviceSynchronize();
cudaMemcpy(hC, dC, sizeof(float) * m * n, cudaMemcpyDeviceToHost);
// deallocate device memory
cudaFree(dA);
cudaFree(dB);
cudaFree(dC);
// DEVICE END //

// RESULTS //
printf("basic: %f, tiled: %f\n", ms_basic, ms_tiled);

// deallocate host memory
free(hA);
free(hB);
free(hC);
}

```

Makefile

all: mmult

```

mmult: mmult.cu
    nvcc mmult.cu -o mmult

```

N = 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
 OUTPUT_FILE = data.txt

```

run: mmult
    @rm -f $(OUTPUT_FILE)
    @for N in $(N); do \
        ./mmult $$N >> $(OUTPUT_FILE); \
    done \

```


Runtime Transcript

N = 1

basic: 0.273408, tiled: 0.025600

N = 2

basic: 0.205824, tiled: 0.025600

N = 3

basic: 0.239616, tiled: 0.026624

N = 4

basic: 0.228352, tiled: 0.025600

N = 5

basic: 0.190464, tiled: 0.026624

N = 6

basic: 0.248832, tiled: 0.023552

N = 7

basic: 0.220160, tiled: 0.026624

N = 8

basic: 0.252928, tiled: 0.037888

N = 9

basic: 0.193536, tiled: 0.082944

N = 10

basic: 0.712704, tiled: 0.480256

N = 11

basic: 10.885120, tiled: 6.356992

N = 12

basic: 131.885056, tiled: 67.147774

N = 13

basic: 1142.087646, tiled: 550.513672

N = 14

basic: 9598.697266, tiled: 4405.593262

rsotto@dgx:~/PA05\$

PA6

Comments

Performing LU factorization once and reusing it for multiple right-hand sides significantly reduces computational cost, since factorization is much more expensive than back substitution. This approach is especially beneficial for large systems where multiple solves with the same coefficient matrix are required.

Runtime Transcript

===== N = 2 =====

Time (LU + solve): 0.115712 ms

Time (solve only): 0.020480 ms

===== N = 4 =====

Time (LU + solve): 0.031744 ms

Time (solve only): 0.020480 ms

===== N = 8 =====

Time (LU + solve): 0.032768 ms

Time (solve only): 0.019456 ms

===== N = 16 =====

Time (LU + solve): 0.029696 ms

Time (solve only): 0.020480 ms

===== N = 32 =====

Time (LU + solve): 0.029696 ms

Time (solve only): 0.018432 ms

===== N = 64 =====

Time (LU + solve): 0.043008 ms

Time (solve only): 0.018432 ms

===== N = 128 =====

Time (LU + solve): 0.065536 ms

Time (solve only): 0.019456 ms

===== N = 256 =====

Time (LU + solve): 0.101376 ms

Time (solve only): 0.016384 ms

```
===== N = 512 =====  
Time (LU + solve): 0.183296 ms  
Time (solve only): 0.016384 ms
```

```
===== N = 1024 =====  
Time (LU + solve): 0.817152 ms  
Time (solve only): 0.016384 ms
```

CUDA Code - lu.c

```
/*  
 * Compile:  
 * nvcc -c -I/usr/local/cuda/include lu.c  
 * gcc -o gpulu lu.o -L/usr/local/cuda/lib64 -lcusolver -lcudart  
 */
```

```
#include <stdio.h>  
#include <stdlib.h>  
#include <cuda_runtime.h>  
#include <cusolverDn.h>
```

```
#define MAX_POW 10 // up to  $N = 2^{10} = 1024$ 
```

```
// Generate Hilbert matrix (column-major)
```

```
void generateHilbert(double* A, int N) {  
    int i, j;  
    for (j = 0; j < N; j++) {  
        for (i = 0; i < N; i++) {  
            A[j*N + i] = 1.0 / (i + j + 1);  
        }  
    }  
}
```

```
// Fill RHS with 1s
```

```
void generateB(double* B, int N) {  
    int i;  
    for (i = 0; i < N; i++) B[i] = 1.0;  
}
```

```
// Add random perturbation to RHS
```

```
void perturbB(double* B, int N) {  
    int i;
```

```

    for (i = 0; i < N; i++) {
        double eps = (double)rand() / RAND_MAX; // 0 < eps < 1
        B[i] += eps;
    }
}

int main() {
    cusolverDnHandle_t cusolverH = NULL;
    cudaStream_t stream = NULL;

    cusolverDnCreate(&cusolverH);
    cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
    cusolverDnSetStream(cusolverH, stream);

    // random seed
    srand(1);

    int p;
    for (p = 1; p <= MAX_POW; p++) {

        int N = 1 << p;
        int lda = N, ldb = N;

        printf("\n===== N = %d =====\n", N);

        // Host allocations
        double* A = (double*)malloc(sizeof(double) * N * N);
        double* B = (double*)malloc(sizeof(double) * N);
        double* B2 = (double*)malloc(sizeof(double) * N);

        generateHilbert(A, N);
        generateB(B, N);

        int i;
        for (i = 0; i < N; i++) B2[i] = B[i];
        perturbB(B2, N);

        // Device allocations
        double *d_A, *d_B;
        int* d_info;
        cudaMalloc((void**)&d_A, sizeof(double) * N * N);
        cudaMalloc((void**)&d_B, sizeof(double) * N);
        cudaMalloc((void**)&d_info, sizeof(int));

```

```

cudaMemcpy(d_A, A, sizeof(double) * N * N, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, B, sizeof(double) * N, cudaMemcpyHostToDevice);

// Workspace
int lwork = 0;
cusolverDnDgetrf_bufferSize(cusolverH, N, N, d_A, lda, &lwork);

double* d_work;
cudaMalloc((void**)&d_work, sizeof(double) * lwork);

cudaEvent_t start1, stop1, start2, stop2;
cudaEventCreate(&start1);
cudaEventCreate(&stop1);
cudaEventCreate(&start2);
cudaEventCreate(&stop2);

float time1, time2;

// ---- FIRST SOLVE (factorization + solve) ----
cudaEventRecord(start1, 0);

cusolverDnDgetrf(cusolverH, N, N, d_A, lda, d_work, NULL, d_info);
cusolverDnDgetrs(cusolverH, CUBLAS_OP_N, N, 1, d_A, lda, NULL, d_B, ldb, d_info);

cudaEventRecord(stop1, 0);
cudaEventSynchronize(stop1);
cudaEventElapsedTime(&time1, start1, stop1);

// ---- SECOND SOLVE (reuse LU) ----
cudaMemcpy(d_B, B2, sizeof(double) * N, cudaMemcpyHostToDevice);

cudaEventRecord(start2, 0);
cusolverDnDgetrs(cusolverH, CUBLAS_OP_N, N, 1, d_A, lda, NULL, d_B, ldb, d_info);
cudaEventRecord(stop2, 0);
cudaEventSynchronize(stop2);
cudaEventElapsedTime(&time2, start2, stop2);

printf("Time (LU + solve): %f ms\n", time1);
printf("Time (solve only): %f ms\n", time2);

// Cleanup
free(A); free(B); free(B2);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_info); cudaFree(d_work);

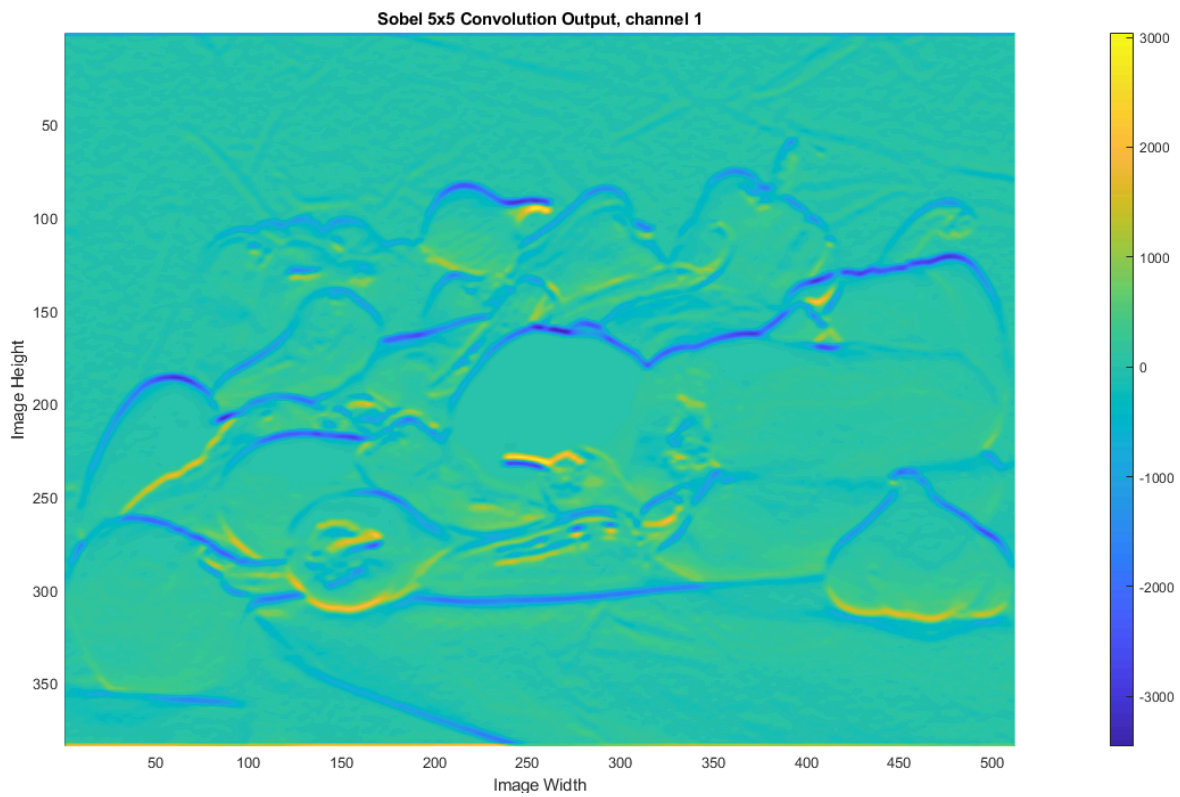
```

```
        cudaEventDestroy(start1); cudaEventDestroy(stop1);  
        cudaEventDestroy(start2); cudaEventDestroy(stop2);  
    }  
  
    cusolverDnDestroy(cusolverH);  
    cudaStreamDestroy(stream);  
    cudaDeviceReset();  
  
    return 0;  
}
```

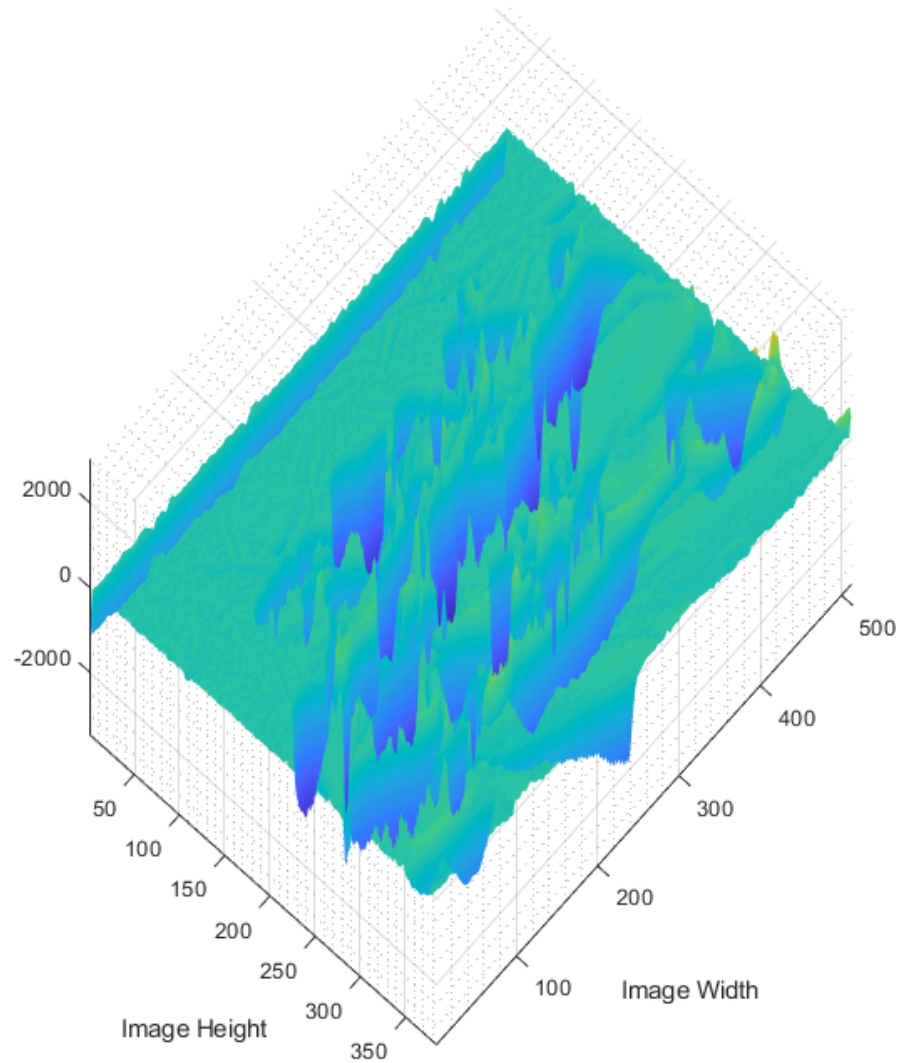

PA7

Comments

peppers.png



Sobel 5x5 Convolution Output, channel 1



CUDA Code - convolution.cu

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
```

```
#define TILE_WIDTH 16
#define MASK_WIDTH 5
```

```

#define MASK_RADIUS 2

// CUDA error checking macro
#define CUDA_CHECK(err) \
    if (err != cudaSuccess) { \
        printf("CUDA error: %s\n", cudaGetErrorString(err)); \
        exit(-1); \
    }

__global__ void convolution(unsigned int *input,
                           int *mask,
                           int *output,
                           int channels,
                           int width,
                           int height)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    // Only compute if inside image
    if (row >= height || col >= width) return;

    // Loop over channels (RGB)
    for (int ch = 0; ch < channels; ch++)
    {
        int sum = 0;

        // Apply 5x5 filter
        for (int i = -MASK_RADIUS; i <= MASK_RADIUS; i++)
        {
            for (int j = -MASK_RADIUS; j <= MASK_RADIUS; j++)
            {
                int r = row + i;
                int c = col + j;

                // Zero padding at borders
                if (r >= 0 && r < height && c >= 0 && c < width)
                {
                    int imageIndex = (r * width + c) * channels + ch;
                    int maskIndex = (i + MASK_RADIUS) * MASK_WIDTH + (j + MASK_RADIUS);

                    sum += input[imageIndex] * mask[maskIndex];
                }
            }
        }
    }
}

```

```

    }

    // Write output
    int outputIndex = (row * width + col) * channels + ch;
    output[outputIndex] = sum;
}
}

int main(int argc, char *argv[])
{
    unsigned int *hostInputImage;
    int *hostOutputImage;

    printf("Reading image data...\n");

    // Open file
    FILE *f = fopen(argv[1], "r");
    if (!f) {
        printf("Error opening input file\n");
        return -1;
    }

    // Read dimensions from dat file
    int imageHeight, imageWidth, imageChannels;
    fscanf(f, "%d %d %d", &imageHeight, &imageWidth, &imageChannels);

    printf("Image size: %d x %d x %d\n", imageHeight, imageWidth, imageChannels);

    // Compute total size
    unsigned int inputLength = imageWidth * imageHeight * imageChannels;

    // Allocate host memory
    hostInputImage = (unsigned int *)malloc(inputLength * sizeof(unsigned int));
    hostOutputImage = (int *)malloc(inputLength * sizeof(int));

    // Read pixel data
    unsigned int pixelValue;
    for (unsigned int i = 0; i < inputLength; i++) {
        fscanf(f, "%d", &pixelValue);
        hostInputImage[i] = pixelValue;
    }
    fclose(f);

    // Sobel 5x5 horizontal mask

```

```
int hostMask[5][5] = {
    { 2, 2, 4, 2, 2},
    { 1, 1, 2, 1, 1},
    { 0, 0, 0, 0, 0},
    {-1, -1, -2, -1, -1},
    {-2, -2, -4, -2, -2}
};
```

```
unsigned int *deviceInputImage;
int *deviceOutputImage;
int *deviceMask;
```

```
// Allocate GPU memory
```

```
CUDA_CHECK(cudaMalloc((void **)&deviceInputImage,
    inputLength * sizeof(unsigned int)));
```

```
CUDA_CHECK(cudaMalloc((void **)&deviceOutputImage,
    inputLength * sizeof(int)));
```

```
CUDA_CHECK(cudaMalloc((void **)&deviceMask,
    MASK_WIDTH * MASK_WIDTH * sizeof(int)));
```

```
// Copy data to GPU
```

```
CUDA_CHECK(cudaMemcpy(deviceInputImage,
    hostInputImage,
    inputLength * sizeof(unsigned int),
    cudaMemcpyHostToDevice));
```

```
CUDA_CHECK(cudaMemcpy(deviceMask,
    hostMask,
    MASK_WIDTH * MASK_WIDTH * sizeof(int),
    cudaMemcpyHostToDevice));
```

```
// Configure execution
```

```
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH, 1);
dim3 dimGrid(
    (imageWidth + TILE_WIDTH - 1) / TILE_WIDTH,
    (imageHeight + TILE_WIDTH - 1) / TILE_WIDTH
);
```

```
// Launch kernel
```

```
convolution<<<dimGrid, dimBlock>>>(
    deviceInputImage,
    deviceMask,
```

```

        deviceOutputImage,
        imageChannels,
        imageWidth,
        imageHeight
    );

    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Copy result back
    CUDA_CHECK(cudaMemcpy(hostOutputImage,
        deviceOutputImage,
        inputLength * sizeof(int),
        cudaMemcpyDeviceToHost));

    // Write output file
    f = fopen("output.out", "w");
    for (unsigned int i = 0; i < inputLength; i++)
        fprintf(f, "%d\n", hostOutputImage[i]);
    fclose(f);

    // Cleanup
    cudaFree(deviceInputImage);
    cudaFree(deviceOutputImage);
    cudaFree(deviceMask);

    free(hostInputImage);
    free(hostOutputImage);

    printf("Done.\n");

    return 0;
}

```

Matlab Code - in.m

```

% Read in original RGB image.
I = imread('peppers.png');
[m,n,c]=size(I);
fileID = fopen('input.dat','w');
% include image dimensions
fprintf(fileID, '%d %d %d\n', m, n, c);
for i = 1:m,

```

```

        for j = 1:n,
            for k = 1:c,
                fprintf(fileID, '%d\n', I(i,j,k));
            end
        end
    end
end
fclose(fileID);

```

Matlab Code - out.m

```

clc
close all
m = 384; % height
n = 512; % width
c = 3;   % channels
P = zeros(m,n,c);
fid = fopen('output.out','r');
i = 1; j = 1; k = 1;
for l = 1:m*n*c,
    p = fscanf(fid, '%d', 1);
    P(i,j,k) = p;
    if mod(k,c) == 0
        k = 1;
        j = j + 1;
        if j == n + 1
            j = 1;
            i = i + 1;
            if i == m + 1
                break;
            end
        end
    end
    else
        k = k + 1;
    end
end
fclose(fid);
[M,N] = meshgrid(1:1:n,1:1:m);
c = 1;
surf(M,N,P(:,:,c));
xlabel('Image Width')
ylabel('Image Height')
title(['Sobel 5x5 Convolution Output, channel ', num2str(c)])
grid on
grid minor
set(gcf, 'color', 'w')
axis equal

```

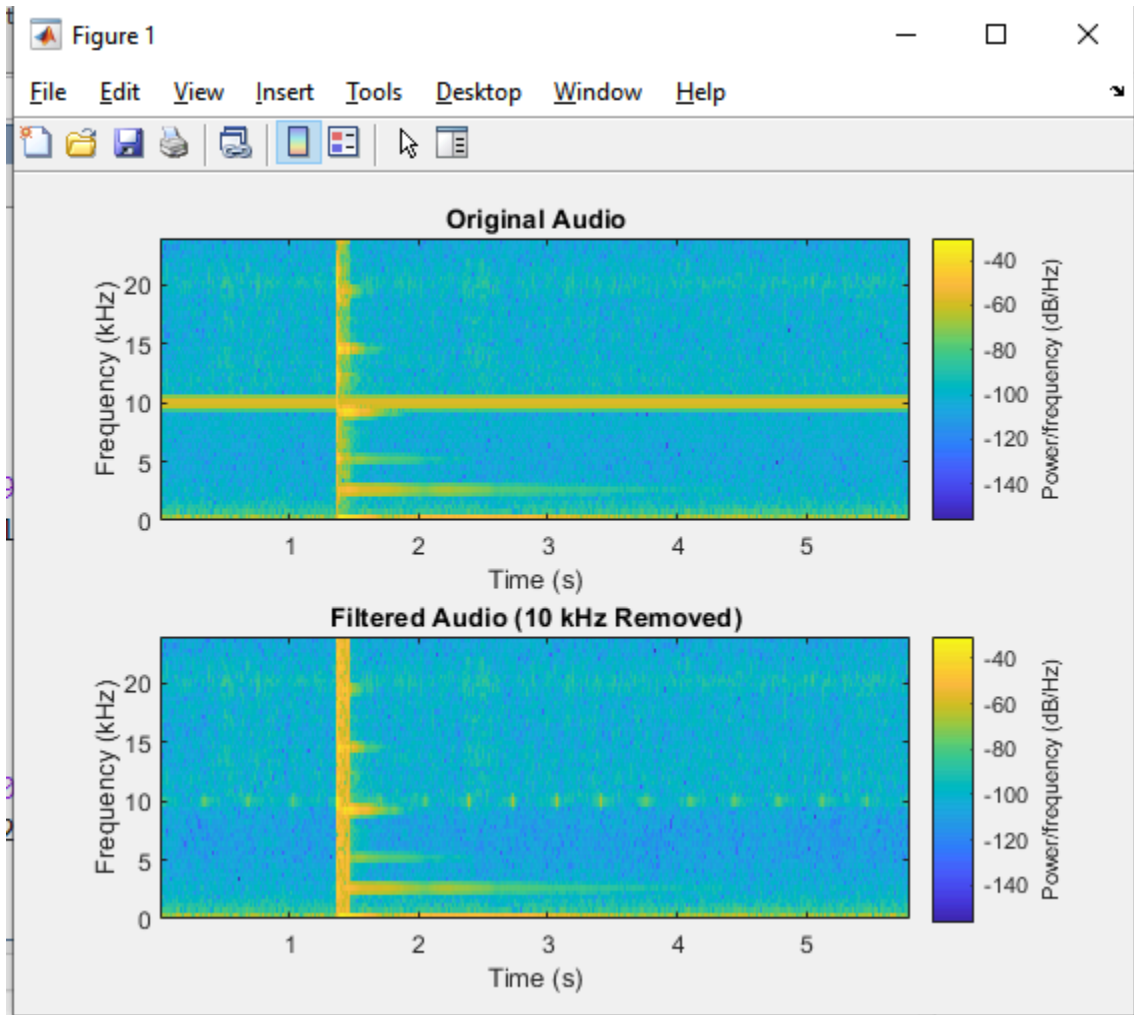


```
colorbar
shading interp
rotate3d
view(0,90);
set(gcf,'position',[10,10,1280,768])
figure(1)
```

PA8

Results

Spectrograms



Output

```
rsotto@dgx:~/PA08$ ./FFTW compe596_mix_48kHz_16bit_mono_Mixdown_1.wav
Target bin (10kHz): 3413

==== FINAL TIMINGS ====
Total CPU time: 3.92857 ms
Total GPU time: 4.30119 ms
rsotto@dgx:~/PA08$
```

Comments

The spectrogram of the filtered audio shows that the strong horizontal component at 10 kHz present in the original signal has been significantly attenuated. The rest of the spectrogram remains largely unchanged, indicating that the notch filter successfully removed the targeted 10kHz frequency without distorting the overall signal. Although the GPU provides faster computation for the FFT itself, data transfer between host (CPU) and device (GPU) memory introduces overhead, likely causing the CPU implementation to surpass the GPU implementation for the given sample. A longer audio sample could allow the GPU implementation to surpass the CPU despite the overhead required.

Source Code - FFTW.cpp

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <chrono>

#include <fftw3.h>
#include <cuFFT.h>
#include <cuda_runtime.h>
#include "WavFile.h"

using namespace std;
using namespace std::chrono;

#define BUFF_SIZE 16384

#define CUDA_CHECK(ans) { gpuAssert((ans), __FILE__, __LINE__); }
inline void gpuAssert(cudaError_t code, const char *file, int line, bool abort=true){
    if(code != cudaSuccess){
        fprintf(stderr,"GPUassert: %s %s %d\n", cudaGetErrorString(code), file, line);
        if(abort) exit(code);
    }
}

// ----- NOTCH FILTER FUNCTION -----
void notchFilter(fftw_complex *data, int N, int targetBin, int width){
    for(int k = targetBin - width; k <= targetBin + width; k++){
        if(k >= 0 && k < N){
            data[k][0] = 0.0;
```

```

        data[k][1] = 0.0;

        int mirror = (N - k) % N;
        data[mirror][0] = 0.0;
        data[mirror][1] = 0.0;
    }
}

void notchFilterGPU(cufftDoubleComplex *data, int N, int targetBin, int width){
    for(int k = targetBin - width; k <= targetBin + width; k++){
        if(k >= 0 && k < N){
            data[k].x = 0.0;
            data[k].y = 0.0;

            int mirror = (N - k) % N;
            data[mirror].x = 0.0;
            data[mirror].y = 0.0;
        }
    }
}

void notchFilterGPU(cufftDoubleComplex *data, int N, int targetBin, int width){
    for(int k = targetBin - width; k <= targetBin + width; k++){
        if(k >= 0 && k < N){
            data[k].x = 0.0;
            data[k].y = 0.0;

            int mirror = (N - k) % N;
            data[mirror].x = 0.0;
            data[mirror].y = 0.0;
        }
    }
}

int main(int argc, char *argv[]) {

    if(argc != 2){
        printf("usage: %s <input.wav>\n", argv[0]);
        return 1;
    }

    const char *wavfile = argv[1];

```

```

WavInFile inFile(wavfile);
WavOutFile outFile("FFTW_P08_out.wav",
    inFile.getSampleRate(),
    inFile.getNumBits(),
    inFile.getNumChannels());

double Fs = inFile.getSampleRate();
double df = Fs / BUFF_SIZE;
int targetBin = (int)round(10000.0 / df);
int width = 20;

cout << "Target bin (10kHz): " << targetBin << endl;

// ----- CPU (FFTW) -----
fftw_complex *in, *out, *ifft_out;
in = (fftw_complex*)fftw_malloc(sizeof(fftw_complex)*BUFF_SIZE);
out = (fftw_complex*)fftw_malloc(sizeof(fftw_complex)*BUFF_SIZE);
ifft_out = (fftw_complex*)fftw_malloc(sizeof(fftw_complex)*BUFF_SIZE);

fftw_plan fft_plan = fftw_plan_dft_1d(BUFF_SIZE, in, out, FFTW_FORWARD,
FFTW_ESTIMATE);
fftw_plan ifft_plan = fftw_plan_dft_1d(BUFF_SIZE, out, ifft_out, FFTW_BACKWARD,
FFTW_ESTIMATE);

// ----- GPU (cuFFT) -----
cufftHandle plan;
cufftPlan1d(&plan, BUFF_SIZE, CUFFT_Z2Z, 1);

cufftDoubleComplex *d_data, *h_data;
cudaMalloc((void**)&d_data, BUFF_SIZE * sizeof(cufftDoubleComplex));
h_data = (cufftDoubleComplex*)malloc(sizeof(cufftDoubleComplex)*BUFF_SIZE);

short sampleBuffer[BUFF_SIZE];
short buffer[BUFF_SIZE];

double total_cpu = 0;
double total_gpu = 0;

while(!inFile.eof()){

    size_t samplesRead = inFile.read(sampleBuffer, BUFF_SIZE);

    // ===== CPU PIPELINE =====
    for(int i=0;i<BUFF_SIZE;i++){

```

```

    in[i][0] = sampleBuffer[i];
    in[i][1] = 0.0;
}

auto cpu_start = high_resolution_clock::now();

fftw_execute(fftw_plan);
notchFilter(out, BUFF_SIZE, targetBin, width);
fftw_execute(ifft_plan);

auto cpu_end = high_resolution_clock::now();
total_cpu += duration<double, milli>(cpu_end - cpu_start).count();

// ===== GPU PIPELINE =====
for(int i=0;i<BUFF_SIZE;i++){
    h_data[i].x = sampleBuffer[i];
    h_data[i].y = 0.0;
}
    CUDA_CHECK(cudaMemcpy(d_data, h_data,
        BUFF_SIZE*sizeof(cufftDoubleComplex), cudaMemcpyHostToDevice));

auto gpu_start = high_resolution_clock::now();

cufftExecZ2Z(plan, d_data, d_data, CUFFT_FORWARD);
CUDA_CHECK(cudaDeviceSynchronize());

CUDA_CHECK(cudaMemcpy(h_data, d_data,
    BUFF_SIZE*sizeof(cufftDoubleComplex), cudaMemcpyDeviceToHost));

notchFilterGPU(h_data, BUFF_SIZE, targetBin, width);

CUDA_CHECK(cudaMemcpy(d_data, h_data,
    BUFF_SIZE*sizeof(cufftDoubleComplex), cudaMemcpyHostToDevice));

cufftExecZ2Z(plan, d_data, d_data, CUFFT_INVERSE);
CUDA_CHECK(cudaDeviceSynchronize());

auto gpu_end = high_resolution_clock::now();
total_gpu += duration<double, milli>(gpu_end - gpu_start).count();
CUDA_CHECK(cudaMemcpy(h_data, d_data,
    BUFF_SIZE*sizeof(cufftDoubleComplex), cudaMemcpyDeviceToHost));

// ----- WRITE OUTPUT (GPU result) -----
for(int i=0;i<BUFF_SIZE;i++){

```

```
        buffer[i] = (short)(h_data[i].x / BUFF_SIZE);
    }

    outFile.write(buffer, BUFF_SIZE);
}

cout << "\n=== FINAL TIMINGS ===\n";
cout << "Total CPU time: " << total_cpu << " ms\n";
cout << "Total GPU time: " << total_gpu << " ms\n";

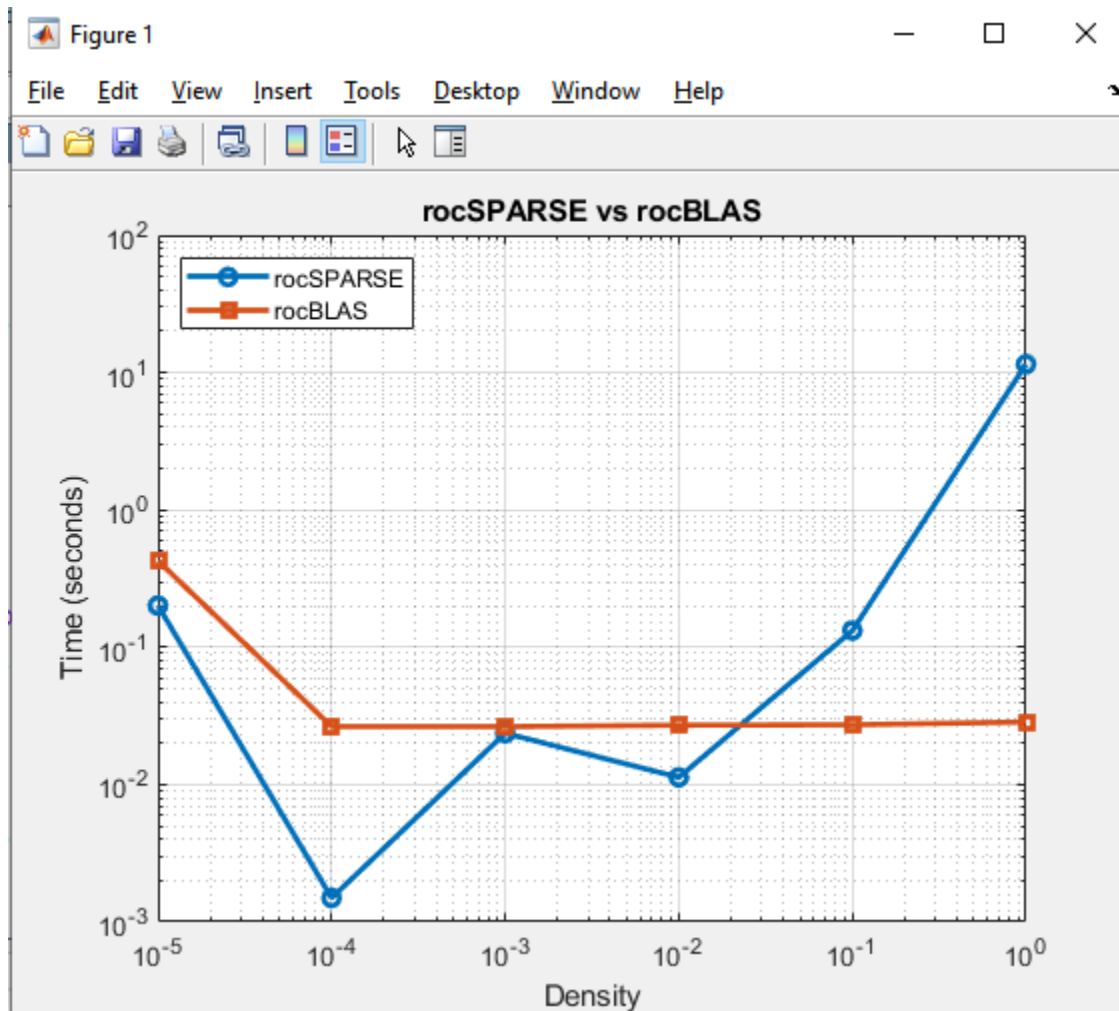
// Cleanup
fftw_destroy_plan(fftw_plan);
fftw_destroy_plan(iffw_plan);
fftw_free(in);
fftw_free(out);
fftw_free(iffw_out);

cufftDestroy(plan);
cudaFree(d_data);
free(h_data);

return 0;
}
```


PA9

Time vs Density Plot



Comments

The results show that rocSPARSE outperforms rocBLAS at low densities, while rocBLAS becomes more efficient as the matrices become denser. The crossover point occurs between densities 10^{-2} and 10^{-1} . At low densities, sparse matrix multiplication is faster because it skips operations on zero elements, reducing computation and memory usage. However, as density increases, the overhead of indexing and irregular memory access reduces the sparse advantage. In contrast, dense matrix multiplication performs a fixed amount of work but benefits from highly optimized GPU implementations, making it faster for moderately dense and fully dense matrices.

HIP Code – csrgemm.cc

```
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstddef>
#include <cstdint>
#include <cstdlib>
#include <cstring>
#include <iomanip>
#include <iostream>
#include <limits>
#include <numeric>
#include <random>
#include <stdexcept>
#include <string>
#include <vector>

#include <hip/hip_runtime.h>
#include <roclblas/roclblas.h>
#include <rocspase/rocspase.h>

#define CHECK_HIP(cmd) \
do \
{ \
    hipError_t _err = (cmd); \
    if(_err != hipSuccess) \
    { \
        std::cerr << "HIP error: " << hipGetErrorString(_err) \
        << " at " << __FILE__ << ":" << __LINE__ << std::endl; \
        std::exit(EXIT_FAILURE); \
    } \
} while(false)

#define CHECK_ROCBLAS(cmd) \
do \
{ \
    roclblas_status _err = (cmd); \
    if(_err != roclblas_status_success) \
    { \
        std::cerr << "rocBLAS error: " << static_cast<int>(_err) \
        << " at " << __FILE__ << ":" << __LINE__ << std::endl; \
        std::exit(EXIT_FAILURE); \
    } \
}
```

```

    } while(false)

#define CHECK_ROCSPARSE(cmd) \
do \
{ \
    rocsparse_status_err = (cmd); \
    if(_err != rocsparse_status_success) \
    { \
        std::cerr << "rocSPARSE error: " << static_cast<int>(_err) \
        << " at " << __FILE__ << ":" << __LINE__ << std::endl; \
        std::exit(EXIT_FAILURE); \
    } \
} while(false)

```

```

namespace
{
    const int N = 1 << 13; // 8192
    const float ALPHA = 1.0f;
    const float BETA = 0.0f;

```

```

    struct CsrMatrix
    {
        int rows = 0;
        int cols = 0;
        std::vector<int> row_ptr;
        std::vector<int> col_ind;
        std::vector<float> val;
    };

```

```

    struct DeviceCsrMatrix
    {
        int rows = 0;
        int cols = 0;
        int nnz = 0;
        int* row_ptr = nullptr;
        int* col_ind = nullptr;
        float* val = nullptr;
    };

```

```

    struct SampleErrorSummary
    {
        double max_abs = 0.0;
        double max_rel = 0.0;
        bool passed = true;
    };

```

```
};
```

```
void free_device_csr(DeviceCsrMatrix& M)
{
    if(M.row_ptr) CHECK_HIP(hipFree(M.row_ptr));
    if(M.col_ind) CHECK_HIP(hipFree(M.col_ind));
    if(M.val) CHECK_HIP(hipFree(M.val));
    M.row_ptr = nullptr;
    M.col_ind = nullptr;
    M.val = nullptr;
    M.nnz = 0;
    M.rows = 0;
    M.cols = 0;
}
```

```
CsrMatrix generate_sparse_matrix_csr(int rows, int cols, double density, std::uint32_t seed)
{
    CsrMatrix A;
    A.rows = rows;
    A.cols = cols;
    A.row_ptr.resize(rows + 1, 0);

    std::mt19937 rng(seed);
    std::uniform_real_distribution<float> prob(0.0f, 1.0f);
    std::uniform_real_distribution<float> val_dist(0.1f, 1.0f);

    for(int i = 0; i < rows; ++i)
    {
        A.row_ptr[i] = static_cast<int>(A.col_ind.size());
        for(int j = 0; j < cols; ++j)
        {
            if(prob(rng) < density)
            {
                A.col_ind.push_back(j);
                A.val.push_back(val_dist(rng));
            }
        }
    }
    A.row_ptr[rows] = static_cast<int>(A.col_ind.size());
    return A;
}
```

```
std::vector<float> csr_to_dense_col_major(const CsrMatrix& A)
{

```

```

std::vector<float> dense(static_cast<std::size_t>(A.rows) * A.cols, 0.0f);
for(int i = 0; i < A.rows; ++i)
{
    for(int p = A.row_ptr[i]; p < A.row_ptr[i + 1]; ++p)
    {
        const int j = A.col_ind[p];
        dense[static_cast<std::size_t>(i) + static_cast<std::size_t>(j) * A.rows] = A.val[p];
    }
}
return dense;
}

```

```

DeviceCsrMatrix copy_csr_to_device(const CsrMatrix& A)
{
    DeviceCsrMatrix dA;
    dA.rows = A.rows;
    dA.cols = A.cols;
    dA.nnz = static_cast<int>(A.val.size());

    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dA.row_ptr), sizeof(int) *
A.row_ptr.size()));
    CHECK_HIP(hipMemcpy(dA.row_ptr,
        A.row_ptr.data(),
        sizeof(int) * A.row_ptr.size(),
        hipMemcpyHostToDevice));

    const std::size_t nz_alloc = std::max<std::size_t>(1, A.val.size());
    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dA.col_ind), sizeof(int) * nz_alloc));
    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dA.val), sizeof(float) * nz_alloc));

    if(!A.val.empty())
    {
        CHECK_HIP(hipMemcpy(dA.col_ind,
            A.col_ind.data(),
            sizeof(int) * A.col_ind.size(),
            hipMemcpyHostToDevice));
        CHECK_HIP(hipMemcpy(dA.val,
            A.val.data(),
            sizeof(float) * A.val.size(),
            hipMemcpyHostToDevice));
    }

    return dA;
}

```

```

float host_csr_value(const CsrMatrix& A, int row, int col)
{
    const int begin = A.row_ptr[row];
    const int end   = A.row_ptr[row + 1];
    const int* first = A.col_ind.data() + begin;
    const int* last  = A.col_ind.data() + end;
    const int* it    = std::lower_bound(first, last, col);
    if(it == last || *it != col)
    {
        return 0.0f;
    }
    return A.val[static_cast<std::size_t>(it - A.col_ind.data())];
}

float sample_reference_entry_from_Acsr_Bdense(const CsrMatrix& A,
                                              const std::vector<float>& B_dense_col_major,
                                              int row,
                                              int col)
{
    double sum = 0.0;
    for(int p = A.row_ptr[row]; p < A.row_ptr[row + 1]; ++p)
    {
        const int k = A.col_ind[p];
        sum += static_cast<double>(A.val[p]) *
               static_cast<double>(B_dense_col_major[static_cast<std::size_t>(k) +
static_cast<std::size_t>(col) * A.cols]);
    }
    return static_cast<float>(sum);
}

SampleErrorSummary validate_dense_samples(const std::vector<float>&
C_dense_col_major,
                                         const CsrMatrix& A,
                                         const std::vector<float>& B_dense_col_major,
                                         int num_samples,
                                         std::uint32_t seed,
                                         double rel_tol,
                                         double abs_tol)
{
    SampleErrorSummary summary;
    std::mt19937 rng(seed);
    std::uniform_int_distribution<int> row_dist(0, A.rows - 1);
    std::uniform_int_distribution<int> col_dist(0, A.cols - 1);

```

```

for(int s = 0; s < num_samples; ++s)
{
    const int row = row_dist(rng);
    const int col = col_dist(rng);
    const float ref = sample_reference_entry_from_Acsr_Bdense(A, B_dense_col_major,
row, col);
    const float got = C_dense_col_major[static_cast<std::size_t>(row) +
static_cast<std::size_t>(col) * A.rows];
    const double abs_err = std::fabs(static_cast<double>(got) - static_cast<double>(ref));
    const double rel_err = abs_err / std::max(1.0, std::fabs(static_cast<double>(ref)));

    summary.max_abs = std::max(summary.max_abs, abs_err);
    summary.max_rel = std::max(summary.max_rel, rel_err);
    if(!(abs_err <= abs_tol || rel_err <= rel_tol))
    {
        summary.passed = false;
    }
}

return summary;
}

```

```

SampleErrorSummary validate_csr_samples(const CsrMatrix& C,
                                     const CsrMatrix& A,
                                     const std::vector<float>& B_dense_col_major,
                                     int num_samples,
                                     std::uint32_t seed,
                                     double rel_tol,
                                     double abs_tol)
{
    SampleErrorSummary summary;
    std::mt19937 rng(seed);
    std::uniform_int_distribution<int> row_dist(0, A.rows - 1);
    std::uniform_int_distribution<int> col_dist(0, A.cols - 1);

    for(int s = 0; s < num_samples; ++s)
    {
        const int row = row_dist(rng);
        const int col = col_dist(rng);
        const float ref = sample_reference_entry_from_Acsr_Bdense(A, B_dense_col_major,
row, col);
        const float got = host_csr_value(C, row, col);
        const double abs_err = std::fabs(static_cast<double>(got) - static_cast<double>(ref));

```



```

const double rel_err = abs_err / std::max(1.0, std::fabs(static_cast<double>(ref)));

summary.max_abs = std::max(summary.max_abs, abs_err);
summary.max_rel = std::max(summary.max_rel, rel_err);
if(!(abs_err <= abs_tol || rel_err <= rel_tol))
{
    summary.passed = false;
}
}

return summary;
}

double sampled_sparse_vs_dense_diff(const CsrMatrix& C_sparse,
                                   const std::vector<float>& C_dense_col_major,
                                   int rows,
                                   int cols,
                                   int num_samples,
                                   std::uint32_t seed)
{
    std::mt19937 rng(seed);
    std::uniform_int_distribution<int> row_dist(0, rows - 1);
    std::uniform_int_distribution<int> col_dist(0, cols - 1);
    double max_abs = 0.0;

    for(int s = 0; s < num_samples; ++s)
    {
        const int row = row_dist(rng);
        const int col = col_dist(rng);
        const double a = static_cast<double>(host_csr_value(C_sparse, row, col));
        const double b = static_cast<double>(C_dense_col_major[static_cast<std::size_t>(row)
+ static_cast<std::size_t>(col) * rows]);
        max_abs = std::max(max_abs, std::fabs(a - b));
    }

    return max_abs;
}

int main()
{
    CHECK_HIP(hipSetDevice(0));

    rocblas_handle handle_blas;

```

```

rocsparse_handle handle_sparse;
CHECK_ROCBLAS(rocblas_create_handle(&handle_blas));
CHECK_ROCSPARSE(rocsparse_create_handle(&handle_sparse));

const double densities[] = {1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1.0};

std::cout << std::fixed << std::setprecision(6);
std::cout << "density,nnzA,nnzB,nnzC,rocSPARSE,rocBLAS\n";

for(int idx = 0; idx < static_cast<int>(sizeof(densities) / sizeof(densities[0])); ++idx)
{
    const double density = densities[idx];

    // Generate A and B in CSR. Distinct seeds keep runs reproducible.
    CsrMatrix A = generate_sparse_matrix_csr(N, N, density, 12345u +
static_cast<std::uint32_t>(idx) * 17u);
    CsrMatrix B = generate_sparse_matrix_csr(N, N, density, 54321u +
static_cast<std::uint32_t>(idx) * 31u);

    // Dense column-major copies for rocBLAS and sampled correctness checks.
    std::vector<float> A_dense = csr_to_dense_col_major(A);
    std::vector<float> B_dense = csr_to_dense_col_major(B);

    DeviceCsrMatrix dA = copy_csr_to_device(A);
    DeviceCsrMatrix dB = copy_csr_to_device(B);

    // Dummy D for beta = 0. rocSPARSE still expects valid descriptors/pointers when beta is a
    valid pointer.
    std::vector<int> hD_row_ptr(N + 1, 0);
    int* dD_row_ptr = nullptr;
    int* dD_col_ind = nullptr;
    float* dD_val = nullptr;
    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dD_row_ptr), sizeof(int) *
hD_row_ptr.size()));
    CHECK_HIP(hipMemcpy(dD_row_ptr,
        hD_row_ptr.data(),
        sizeof(int) * hD_row_ptr.size(),
        hipMemcpyHostToDevice));
    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dD_col_ind), sizeof(int)));
    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dD_val), sizeof(float)));
    CHECK_HIP(hipMemset(dD_col_ind, 0, sizeof(int)));
    CHECK_HIP(hipMemset(dD_val, 0, sizeof(float)));

    rocsparse_mat_descr descrA, descrB, descrC, descrD;

```

```

CHECK_ROCSPARSE(rocsparse_create_mat_descr(&descrA));
CHECK_ROCSPARSE(rocsparse_create_mat_descr(&descrB));
CHECK_ROCSPARSE(rocsparse_create_mat_descr(&descrC));
CHECK_ROCSPARSE(rocsparse_create_mat_descr(&descrD));
CHECK_ROCSPARSE(rocsparse_set_mat_index_base(descrA,
rocsparse_index_base_zero));
CHECK_ROCSPARSE(rocsparse_set_mat_index_base(descrB,
rocsparse_index_base_zero));
CHECK_ROCSPARSE(rocsparse_set_mat_index_base(descrC,
rocsparse_index_base_zero));
CHECK_ROCSPARSE(rocsparse_set_mat_index_base(descrD,
rocsparse_index_base_zero));
CHECK_ROCSPARSE(rocsparse_set_mat_type(descrA,
rocsparse_matrix_type_general));
CHECK_ROCSPARSE(rocsparse_set_mat_type(descrB,
rocsparse_matrix_type_general));
CHECK_ROCSPARSE(rocsparse_set_mat_type(descrC,
rocsparse_matrix_type_general));
CHECK_ROCSPARSE(rocsparse_set_mat_type(descrD,
rocsparse_matrix_type_general));

rocsparse_mat_info infoC;
CHECK_ROCSPARSE(rocsparse_create_mat_info(&infoC));

size_t buffer_size = 0;
CHECK_ROCSPARSE(rocsparse_scsrgermm_buffer_size(handle_sparse,
rocsparse_operation_none,
rocsparse_operation_none,
N,
N,
N,
&ALPHA,
descrA,
dA.nnz,
dA.row_ptr,
dA.col_ind,
descrB,
dB.nnz,
dB.row_ptr,
dB.col_ind,
&BETA,
descrD,
0,
dD_row_ptr,

```

```

        dD_col_ind,
        infoC,
        &buffer_size));

void* temp_buffer = nullptr;
CHECK_HIP(hipMalloc(&temp_buffer, std::max<std::size_t>(buffer_size, 1)));

int* dC_row_ptr = nullptr;
int* dC_col_ind = nullptr;
float* dC_val = nullptr;
CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dC_row_ptr), sizeof(int) * (N + 1)));

// Time the sparse multiplication path. This includes the required nnz discovery step
// plus rocsparse_csrgemm itself, but excludes host-device transfers and matrix
generation.
int nnzC = 0;
auto sparse_start = std::chrono::high_resolution_clock::now();

CHECK_ROCSPARSE(rocsparse_csrgemm_nnz(handle_sparse,
        rocsparse_operation_none,
        rocsparse_operation_none,
        N,
        N,
        N,
        descrA,
        dA.nnz,
        dA.row_ptr,
        dA.col_ind,
        descrB,
        dB.nnz,
        dB.row_ptr,
        dB.col_ind,
        descrD,
        0,
        dD_row_ptr,
        dD_col_ind,
        descrC,
        dC_row_ptr,
        &nnzC,
        infoC,
        temp_buffer));

CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dC_col_ind), sizeof(int) * std::max(1,
nnzC)));

```

```
CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dC_val), sizeof(float) * std::max(1,
nnzC)));
```

```
CHECK_ROCSPARSE(rocparse_scsrgemm(handle_sparse,
    rocparse_operation_none,
    rocparse_operation_none,
    N,
    N,
    N,
    &ALPHA,
    descrA,
    dA.nnz,
    dA.val,
    dA.row_ptr,
    dA.col_ind,
    descrB,
    dB.nnz,
    dB.val,
    dB.row_ptr,
    dB.col_ind,
    &BETA,
    descrD,
    0,
    dD_val,
    dD_row_ptr,
    dD_col_ind,
    descrC,
    dC_val,
    dC_row_ptr,
    dC_col_ind,
    infoC,
    temp_buffer));
```

```
CHECK_HIP(hipDeviceSynchronize());
auto sparse_end = std::chrono::high_resolution_clock::now();
const double sparse_time = std::chrono::duration<double>(sparse_end -
sparse_start).count();
```

```
// Bring sparse result back to host for verification.
CsrMatrix C_sparse;
C_sparse.rows = N;
C_sparse.cols = N;
C_sparse.row_ptr.resize(N + 1);
C_sparse.col_ind.resize(std::max(0, nnzC));
```

```

    C_sparse.val.resize(std::max(0, nnzC));
    CHECK_HIP(hipMemcpy(C_sparse.row_ptr.data(), dC_row_ptr, sizeof(int) * (N + 1),
hipMemcpyDeviceToHost));
    if(nnzC > 0)
    {
        CHECK_HIP(hipMemcpy(C_sparse.col_ind.data(), dC_col_ind, sizeof(int) * nnzC,
hipMemcpyDeviceToHost));
        CHECK_HIP(hipMemcpy(C_sparse.val.data(), dC_val, sizeof(float) * nnzC,
hipMemcpyDeviceToHost));
    }

    // rocBLAS path with dense matrices in column-major layout.
    float* dA_dense = nullptr;
    float* dB_dense = nullptr;
    float* dC_dense = nullptr;
    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dA_dense), sizeof(float) *
static_cast<std::size_t>(N) * N));
    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dB_dense), sizeof(float) *
static_cast<std::size_t>(N) * N));
    CHECK_HIP(hipMalloc(reinterpret_cast<void**>(&dC_dense), sizeof(float) *
static_cast<std::size_t>(N) * N));
    CHECK_HIP(hipMemcpy(dA_dense,
        A_dense.data(),
        sizeof(float) * static_cast<std::size_t>(N) * N,
        hipMemcpyHostToDevice));
    CHECK_HIP(hipMemcpy(dB_dense,
        B_dense.data(),
        sizeof(float) * static_cast<std::size_t>(N) * N,
        hipMemcpyHostToDevice));
    CHECK_HIP(hipMemset(dC_dense, 0, sizeof(float) * static_cast<std::size_t>(N) * N));

    auto dense_start = std::chrono::high_resolution_clock::now();
    CHECK_ROCBLAS(rocblas_sgemm(handle_blas,
        rocblas_operation_none,
        rocblas_operation_none,
        N,
        N,
        N,
        &ALPHA,
        dA_dense,
        N,
        dB_dense,
        N,
        &BETA,

```

```

        dC_dense,
        N));
CHECK_HIP(hipDeviceSynchronize());
auto dense_end = std::chrono::high_resolution_clock::now();
const double dense_time = std::chrono::duration<double>(dense_end -
dense_start).count();

std::vector<float> C_dense(static_cast<std::size_t>(N) * N);
CHECK_HIP(hipMemcpy(C_dense.data(),
        dC_dense,
        sizeof(float) * static_cast<std::size_t>(N) * N,
        hipMemcpyDeviceToHost));

const double rel_tol = 1e-3;
const double abs_tol = 5e-2;
const int num_check_samples = 32;

SampleErrorSummary sparse_check = validate_csr_samples(C_sparse,
        A,
        B_dense,
        num_check_samples,
        9001u + static_cast<std::uint32_t>(idx),
        rel_tol,
        abs_tol);

SampleErrorSummary dense_check = validate_dense_samples(C_dense,
        A,
        B_dense,
        num_check_samples,
        1337u + static_cast<std::uint32_t>(idx),
        rel_tol,
        abs_tol);

const double sparse_dense_sample_abs_diff = sampled_sparse_vs_dense_diff(C_sparse,
        C_dense,
        N,
        N,
        num_check_samples,
        777u + static_cast<std::uint32_t>(idx));

// Print results
std::cout << density << ", "
        << dA.nnz << ", "
        << dB.nnz << ", "

```

```

    << nnzC << ", "
    << sparse_time << ", "
    << dense_time << "\n";

// Cleanup this iteration.
CHECK_HIP(hipFree(dA_dense));
CHECK_HIP(hipFree(dB_dense));
CHECK_HIP(hipFree(dC_dense));

CHECK_HIP(hipFree(dC_row_ptr));
CHECK_HIP(hipFree(dC_col_ind));
CHECK_HIP(hipFree(dC_val));
CHECK_HIP(hipFree(temp_buffer));

CHECK_HIP(hipFree(dD_row_ptr));
CHECK_HIP(hipFree(dD_col_ind));
CHECK_HIP(hipFree(dD_val));

free_device_csr(dA);
free_device_csr(dB);

CHECK_ROCSPARSE(rocsparse_destroy_mat_info(infoC));
CHECK_ROCSPARSE(rocsparse_destroy_mat_descr(descrA));
CHECK_ROCSPARSE(rocsparse_destroy_mat_descr(descrB));
CHECK_ROCSPARSE(rocsparse_destroy_mat_descr(descrC));
CHECK_ROCSPARSE(rocsparse_destroy_mat_descr(descrD));
}

CHECK_ROCSPARSE(rocsparse_destroy_handle(handle_sparse));
CHECK_ROCBLAS(rocbblas_destroy_handle(handle_blas));

return 0;
}

```


Matlab Code – csrgemm_plot.m

```
close all; clear; clc;
data = readmatrix('data.csv');
density = data(:,1);
rocSPARSE = data(:,5);
rocBLAS = data(:,6);
figure;
loglog(density, rocSPARSE, '-o', 'LineWidth', 2); hold on;
loglog(density, rocBLAS, '-s', 'LineWidth', 2);
xlabel('Density');
ylabel('Time (seconds)');
title('rocSPARSE vs rocBLAS');
legend('rocSPARSE','rocBLAS','Location','northwest');
grid on;
```

PA10

Host.cpp

```
/**
 * Copyright (C) 2019-2021 Xilinx, Inc
 *
 * Licensed under the Apache License, Version 2.0 (the "License"). You may
 * not use this file except in compliance with the License. A copy of the
 * License is located at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS, WITHOUT
 * WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the
 * License for the specific language governing permissions and limitations
 * under the License.
 */

#include "cmdlineparser.h"
#include <iostream>
#include <cstring>

// XRT includes
#include "xrt/xrt_bo.h"
#include "xrt/xrt_device.h"
#include "xrt/xrt_kernel.h"

// DATA_SIZE MUST BE A MULTIPLE OF 3 (changed from 4096 to 4095)
#define DATA_SIZE 4095

int main(int argc, char** argv) {
    // Command Line Parser
    sda::utils::CmdLineParser parser;

    // Switches
    //*****/"<Full Arg>", "<Short Arg>", "<Description>", "<Default>"
    parser.addSwitch("--xclbin_file", "-x", "input binary file string", "");
    parser.addSwitch("--device_id", "-d", "device index", "0");
    parser.parse(argc, argv);

    // Read settings
    std::string binaryFile = parser.value("xclbin_file");
    int device_index = stoi(parser.value("device_id"));
```

```

if (argc < 3) {
    parser.printHelp();
    return EXIT_FAILURE;
}

std::cout << "Open the device" << device_index << std::endl;
auto device = xrt::device(device_index);
std::cout << "Load the xclbin " << binaryFile << std::endl;
auto uuid = device.load_xclbin(binaryFile);

size_t vector_size_bytes = sizeof(int) * DATA_SIZE;

auto krnl = xrt::kernel(device, uuid, "vadd");

std::cout << "Allocate Buffer in Global Memory\n";
auto bo0 = xrt::bo(device, vector_size_bytes, krnl.group_id(0));
auto bo1 = xrt::bo(device, vector_size_bytes, krnl.group_id(1));
auto bo2 = xrt::bo(device, vector_size_bytes, krnl.group_id(2));
auto bo_out = xrt::bo(device, vector_size_bytes, krnl.group_id(3));

// Map the contents of the buffer object into host memory
auto bo0_map = bo0.map<int*>();
auto bo1_map = bo1.map<int*>();
auto bo2_map = bo2.map<int*>();
auto bo_out_map = bo_out.map<int*>();
std::fill(bo0_map, bo0_map + DATA_SIZE, 0);
std::fill(bo1_map, bo1_map + DATA_SIZE, 0);
std::fill(bo2_map, bo2_map + DATA_SIZE, 0);
std::fill(bo_out_map, bo_out_map + DATA_SIZE, 0);

// Create the test data
int bufReference[DATA_SIZE];
for (int i = 0; i < DATA_SIZE; ++i) {
    bo0_map[i] = i + 1;
    bo1_map[i] = 2 * i + 3;
    bo2_map[i] = 3 * i + 5;
    // calculate triple cross product as reference
    if (i % 3 == 2) {
        int a1 = bo0_map[i-2], a2 = bo0_map[i-1], a3 = bo0_map[i];
        int b1 = bo1_map[i-2], b2 = bo1_map[i-1], b3 = bo1_map[i];
        int c1 = bo2_map[i-2], c2 = bo2_map[i-1], c3 = bo2_map[i];

        int t1 = (b2 * c3 - b3 * c2);
    }
}

```

```

    int t2 = (b3 * c1 - b1 * c3);
    int t3 = (b1 * c2 - b2 * c1);

    bufReference[i-2] = (a2 * t3 - a3 * t2);
    bufReference[i-1] = (a3 * t1 - a1 * t3);
    bufReference[i] = (a1 * t2 - a2 * t1);
}
}

// Synchronize buffer content with device side
std::cout << "synchronize input buffer data to device global memory\n";

bo0.sync(XCL_BO_SYNC_BO_TO_DEVICE);
bo1.sync(XCL_BO_SYNC_BO_TO_DEVICE);
bo2.sync(XCL_BO_SYNC_BO_TO_DEVICE);

std::cout << "Execution of the kernel\n";
auto run = krnl(bo0, bo1, bo2, bo_out, DATA_SIZE);
run.wait();

// Get the output;
std::cout << "Get the output data from the device" << std::endl;
bo_out.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

// Validate our results
if (std::memcmp(bo_out_map, bufReference, DATA_SIZE * sizeof(int)))
    throw std::runtime_error("Value read back does not match reference");

std::cout << "TEST PASSED\n";
return 0;
}

```

Vadd.cpp

```
/**
 * Copyright (C) 2019-2021 Xilinx, Inc
 *
 * Licensed under the Apache License, Version 2.0 (the "License"). You may
 * not use this file except in compliance with the License. A copy of the
 * License is located at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS, WITHOUT
 * WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the
 * License for the specific language governing permissions and limitations
 * under the License.
 */
```

```
/******
```

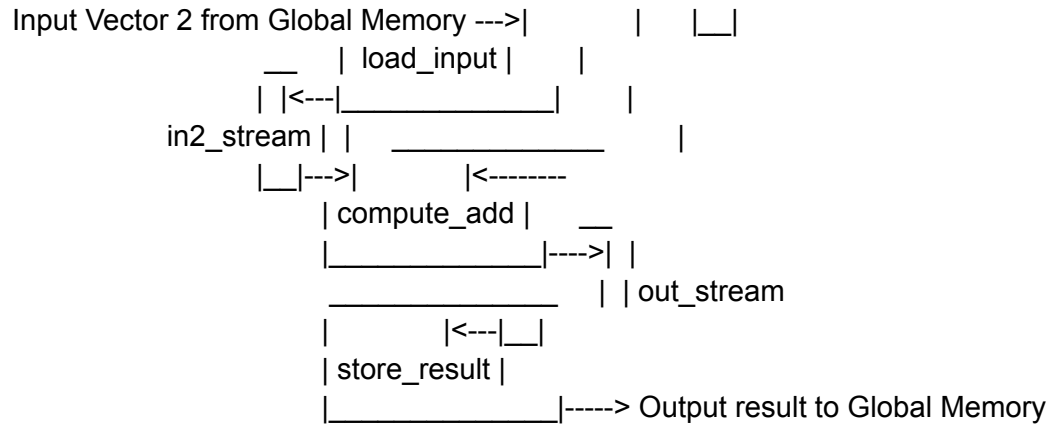
Description:

This example uses the load/compute/store coding style, which is generally the most efficient for implementing kernels using HLS. The load and store functions are responsible for moving data in and out of the kernel as efficiently as possible. The core functionality is decomposed across one of more compute functions. Whenever possible, the compute function should pass data through HLS streams and should contain a single set of nested loops. HLS stream objects are used to pass data between producer and consumer functions. Stream read and write operations have a blocking behavior which allows consumers and producers to synchronize with each other automatically. The dataflow pragma instructs the compiler to enable task-level pipelining. This is required for to load/compute/store functions to execute in a parallel and pipelined manner.

The kernel operates on vectors of NUM_WORDS integers modeled using the `hls::vector` data type. This datatype provides intuitive support for parallelism and fits well the vector-add computation. The vector length is set to NUM_WORDS since NUM_WORDS integers amount to a total of 64 bytes, which is the maximum size of a kernel port. It is a good practice to match the compute bandwidth to the I/O bandwidth. Here the kernel loads, computes and stores NUM_WORDS integer values per clock cycle and is implemented as below:

```

|          |<----- Input Vector 1 from Global Memory
| load_input | _____
|          |----->| |
|          |          | | in1_stream
```



*****/

```
#include <stdint.h>
```

```
#include <hls_stream.h>
```

```
#define DATA_SIZE 4095
```

```
// TRIPCOUNT identifier
```

```
const int c_size = DATA_SIZE;
```

```
static void read_input(int* in, hls::stream<int>& inStream, int size) {
```

```
// Auto-pipeline is going to apply pipeline to this loop
```

```
mem_rd:
```

```
    for (int i = 0; i < size; i++) {
```

```
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
```

```
    inStream << in[i];
```

```
    }
```

```
}
```

```
static void compute_cross_product(hls::stream<int>& inStream1,
```

```
    hls::stream<int>& inStream2,
```

```
    hls::stream<int>& outStream,
```

```
    int size) {
```

```
// Auto-pipeline is going to apply pipeline to this loop
```

```
execute:
```

```
    for (int i = 0; i < size/3; i++) {
```

```
#pragma HLS LOOP_TRIPCOUNT min = c_size/3 max = c_size/3
```

```
    int a1 = inStream1.read(), a2 = inStream1.read(), a3 = inStream1.read();
```

```
    int b1 = inStream2.read(), b2 = inStream2.read(), b3 = inStream2.read();
```

```
    outStream << (a2 * b3 - a3 * b2);
```

```
    outStream << (a3 * b1 - a1 * b3);
```

```

        outStream << (a1 * b2 - a2 * b1);
    }
}

```

```

static void write_result(int* out, hls::stream<int>& outStream, int size) {
// Auto-pipeline is going to apply pipeline to this loop
mem_wr:
    for (int i = 0; i < size; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
        out[i] = outStream.read();
    }
}

```

```

extern "C" {
/*
    Vector Triple Cross Product Kernel Implementation using dataflow
    Arguments:
        in1  (input) --> Input Vector 1
        in2  (input) --> Input Vector 2
        in3  (input) --> Input Vector 3
        out  (output) --> Output Vector
        size (input) --> Size of Vector in Integer (multiple of 3 for 3D Vectors)
*/
void vadd(int* in1, int* in2, int* in3, int* out, int size) {
    static hls::stream<int> inStream1("input_stream_1");
    static hls::stream<int> inStream2("input_stream_2");
    static hls::stream<int> inStream3("input_stream_3");
    static hls::stream<int> tempStream("temp_stream");
    static hls::stream<int> outStream("output_stream");

#pragma HLS INTERFACE m_axi port = in1 bundle = gmem0
#pragma HLS INTERFACE m_axi port = in2 bundle = gmem1
#pragma HLS INTERFACE m_axi port = in3 bundle = gmem2
#pragma HLS INTERFACE m_axi port = out bundle = gmem0

#pragma HLS dataflow
    // temporary data stream to hold first cross product result

    // dataflow pragma instruct compiler to run following three APIs in parallel
    read_input(in1, inStream1, size);
    read_input(in2, inStream2, size);
    read_input(in3, inStream3, size);

    //compute_add(inStream1, inStream2, outStream, size);

```



```
// compute first cross product between in2 and in3
compute_cross_product(inStream2, inStream3, tempStream, size);
// compute second cross product between in1 and (in2 x in3)
compute_cross_product(inStream1, tempStream, outStream, size);

write_result(out, outStream, size);
}
}
```

=== HW_EMU BUILD ===

```
g++ -o hello_world_xrt
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/common/includes/cmdparser/cmdlineparser.cpp
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/common/includes/logger/logger.cpp src/host.cpp
-O2 -pipe -g -feliminate-unused-debug-types -I/opt/xilinx/xrt/include
-I/tools/Xilinx/Vivado/2022.2/include -Wall -O0 -g -std=c++17
-I/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/common/includes/cmdparser
-I/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/common/includes/logger -fmessage-length=0
-Wl,-O1 -Wl,--hash-style=gnu -Wl,--as-needed -Wl,-z,relro,-z,now -L/opt/xilinx/xrt/lib -pthread
-LOpenCL -lrt -lstdc++ -luuid -lxrt_coreutil
mkdir -p ./_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1
v++ -c -g --save-temps -t hw_emu --platform
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_2
02211_1.xpfm -k vadd --temp_dir ./_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1 -l'src'
-o'_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo' 'src/vadd.cpp'
Option Map File Used: '/tools/Xilinx/Vitis/2022.2/data/vitis/vpp/optMap.xml'
```

```
***** v++ v2022.2 (64-bit)
**** SW Build 3671529 on 2022-10-13-17:52:11
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
```

INFO: [v++ 60-1306] Additional information associated with this v++ compile can be found at:

Reports:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/reports/vadd

Log files:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/logs/vadd

Running Dispatch Server on port: 45351

INFO: [v++ 60-1548] Creating build summary session with primary output

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/vadd.xo.compile_summary, at Sun Apr 26 16:43:43 2026

INFO: [v++ 60-1315] Creating rulecheck session with output

'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/reports/vadd/v++_compile_vadd_guidance.html', at Sun Apr 26 16:43:43
2026

INFO: [v++ 60-895] Target platform:

/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_2
02211_1.xpfm

INFO: [v++ 60-1578] This platform contains Xilinx Shell Archive

'/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw/hw.xsa'

INFO: [v++ 74-78] Compiler Version string: 2022.2

INFO: [v++ 60-585] Compiling for hardware emulation target

INFO: [v++ 60-423] Target device: xilinx_u280_gen3x16_xdma_1_202211_1

INFO: [v++ 60-242] Creating kernel: 'vadd'

==>The following messages were generated while performing high-level synthesis for kernel:
vadd Log file:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd/vadd/vitis_hls.log :

INFO: [v++ 204-61] Pipelining loop 'mem_rd'.

INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 3, loop 'mem_rd'

INFO: [v++ 204-61] Pipelining loop 'mem_rd'.

INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 3, loop 'mem_rd'

INFO: [v++ 204-61] Pipelining loop 'mem_rd'.

INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 3, loop 'mem_rd'

INFO: [v++ 204-61] Pipelining loop 'execute'.

INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 3, Depth = 9, loop 'execute'

INFO: [v++ 204-61] Pipelining loop 'execute'.

INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 3, Depth = 9, loop 'execute'

INFO: [v++ 204-61] Pipelining loop 'mem_wr'.

INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 3, loop 'mem_wr'

INFO: [v++ 200-789] **** Estimated Fmax: 411.00 MHz

INFO: [v++ 60-594] Finished kernel compilation

INFO: [v++ 60-244] Generating system estimate report...

INFO: [v++ 60-1092] Generated system estimate report:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/reports/vadd/system_estimate_vadd.txt

WARNING: [v++-17-1525]

INFO: [v++ 60-586] Created _x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo

INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the relevant reports.

Run the following command.

vitis_analyzer

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo.compile_summary

INFO: [v++ 60-791] Total elapsed time: 0h 1m 0s

INFO: [v++ 60-1653] Closing dispatch client.

mkdir -p ./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1

v++ -l -g --save-temps -t hw_emu --platform

/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm --temp_dir ./_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1

-o'./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin'

_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo

Option Map File Used: '/tools/Xilinx/Vitis/2022.2/data/vitis/vpp/optMap.xml'

***** v++ v2022.2 (64-bit)

**** SW Build 3671529 on 2022-10-13-17:52:11

** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.

INFO: [v++ 60-1306] Additional information associated with this v++ link can be found at:

Reports:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link

Log files:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/logs/link

Running Dispatch Server on port: 44089

INFO: [v++ 60-1548] Creating build summary session with primary output

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin.link_summary, at Sun Apr 26 16:44:47 2026

INFO: [v++ 60-1315] Creating rulecheck session with output

'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link/v++_link_vadd.link_guidance.html', at Sun Apr 26 16:44:47 2026

INFO: [v++ 60-895] Target platform:

/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm

INFO: [v++ 60-1578] This platform contains Xilinx Shell Archive

'/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw_emu/hw_emu.xsa'

INFO: [v++ 74-78] Compiler Version string: 2022.2

INFO: [v++ 60-629] Linking for hardware emulation target

INFO: [v++ 60-423] Target device: xilinx_u280_gen3x16_xdma_1_202211_1

INFO: [v++ 60-1332] Run 'run_link' status: Not started

INFO: [v++ 60-1443] [16:44:51] Run run_link: Step system_link: Started

INFO: [v++ 60-1453] Command Line: system_link --xo

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo -keep --xpfm

/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm --target emu --output_dir

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int --temp_dir

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link

INFO: [v++ 60-1454] Run Directory:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link

INFO: [SYSTEM_LINK 82-70] Extracting xo v3 file

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo

INFO: [SYSTEM_LINK 82-53] Creating IP database

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/_sysl/.cdb/xd_ip_db.xml

INFO: [SYSTEM_LINK 82-38] [16:44:53] build_xd_ip_db started:
/tools/Xilinx/Vitis/2022.2/bin/build_xd_ip_db -ip_search 0 -sds-pf
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/hw_emu.hpfm -clkid 0 -ip
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/iprepo/xilinx_com_hls_vadd_1_0.vadd -o
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/_sysl/.cdb/xd_ip_db.xml
INFO: [SYSTEM_LINK 82-37] [16:44:57] build_xd_ip_db finished successfully
Time (s): cpu = 00:00:04 ; elapsed = 00:00:04 . Memory (MB): peak = 430.527 ; gain = 0.000 ;
free physical = 305101 ; free virtual = 464928
INFO: [SYSTEM_LINK 82-51] Create system connectivity graph
INFO: [SYSTEM_LINK 82-102] Applying explicit connections to the system connectivity graph:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/cfgraph/cfgen_cfgraph.xml
INFO: [SYSTEM_LINK 82-38] [16:44:57] cfgen started: /tools/Xilinx/Vitis/2022.2/bin/cfgen
-dpa_mem_offload false -dmclkid 0 -r
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/_sysl/.cdb/xd_ip_db.xml -o
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/cfgraph/cfgen_cfgraph.xml
INFO: [CFGGEN 83-0] Kernel Specs:
INFO: [CFGGEN 83-0] kernel: vadd, num: 1 {vadd_1}
INFO: [CFGGEN 83-2226] Inferring mapping for argument vadd_1.in1 to HBM[0]
INFO: [CFGGEN 83-2226] Inferring mapping for argument vadd_1.out to HBM[0]
INFO: [CFGGEN 83-2226] Inferring mapping for argument vadd_1.in2 to HBM[0]
INFO: [CFGGEN 83-2226] Inferring mapping for argument vadd_1.in3 to HBM[0]
INFO: [SYSTEM_LINK 82-37] [16:45:03] cfgen finished successfully
Time (s): cpu = 00:00:06 ; elapsed = 00:00:06 . Memory (MB): peak = 430.527 ; gain = 0.000 ;
free physical = 302379 ; free virtual = 462201
INFO: [SYSTEM_LINK 82-52] Create top-level block diagram
INFO: [SYSTEM_LINK 82-38] [16:45:03] cf2bd started: /tools/Xilinx/Vitis/2022.2/bin/cf2bd
--linux --trace_buffer 1024 --input_file
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/cfgraph/cfgen_cfgraph.xml --ip_db
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/_sysl/.cdb/xd_ip_db.xml --cf_name dr --working_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/_sysl/.xsd --temp_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link --output_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/int

INFO: [CF2BD 82-31] Launching cf2xd: cf2xd -linux -trace-buffer 1024 -i
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/cfgraph/cfgen_cfgraph.xml -r
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/_sysl/.cdb/xd_ip_db.xml -o dr.xml
INFO: [CF2BD 82-28] cf2xd finished successfully
INFO: [CF2BD 82-31] Launching cf_xsd: cf_xsd -disable-address-gen -dn dr -dp
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/sys_link/_sysl/.xsd
INFO: [CF2BD 82-28] cf_xsd finished successfully
INFO: [SYSTEM_LINK 82-37] [16:45:07] cf2bd finished successfully
Time (s): cpu = 00:00:03 ; elapsed = 00:00:04 . Memory (MB): peak = 430.527 ; gain = 0.000 ;
free physical = 300398 ; free virtual = 460210
INFO: [v++ 60-1441] [16:45:07] Run run_link: Step system_link: Completed
Time (s): cpu = 00:00:15 ; elapsed = 00:00:15 . Memory (MB): peak = 436.348 ; gain = 0.000 ;
free physical = 300455 ; free virtual = 460268
INFO: [v++ 60-1443] [16:45:07] Run run_link: Step cf2sw: Started
INFO: [v++ 60-1453] Command Line: cf2sw -sdsl
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/int/sdsl.dat -rtd
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/int/cf2sw.rtd -nofilter
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/int/cf2sw_full.rtd -xclbin
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/int/xclbin_orig.xml -o
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/int/xclbin_orig.1.xml
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/run_link
INFO: [v++ 60-1441] [16:45:11] Run run_link: Step cf2sw: Completed
Time (s): cpu = 00:00:04 ; elapsed = 00:00:04 . Memory (MB): peak = 436.348 ; gain = 0.000 ;
free physical = 298323 ; free virtual = 458142
INFO: [v++ 60-1443] [16:45:11] Run run_link: Step rtd2_system_diagram: Started
INFO: [v++ 60-1453] Command Line: rtd2SystemDiagram
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_x
dma_1_202211_1/link/run_link
INFO: [v++ 60-1441] [16:45:11] Run run_link: Step rtd2_system_diagram: Completed
Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.29 . Memory (MB): peak = 436.348 ; gain = 0.000
; free physical = 298237 ; free virtual = 458056
INFO: [v++ 60-1443] [16:45:11] Run run_link: Step vpl: Started

INFO: [v++ 60-1453] Command Line: vpl -t hw_emu -f
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm -s -g --remote_ip_cache
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/.ipcache --output_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int --log_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/logs/link --report_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link --config
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vplConfig.ini -k
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/kernel_info.dat --webtalk_flag Vitis --temp_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link --emulation_mode debug_waveform --no-info --iprepo
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/xo/ip_repo/xilinx_com_hls_vadd_1_0 --messageDb
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link/vpl.pb
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/dr.bd.tcl
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link

***** vpl v2022.2 (64-bit)

**** SW Build 3671529 on 2022-10-13-17:52:11

** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.

INFO: [VPL 60-839] Read in kernel information from file
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/kernel_info.dat'.
INFO: [VPL 74-78] Compiler Version string: 2022.2
INFO: [VPL 60-423] Target device: xilinx_u280_gen3x16_xdma_1_202211_1
INFO: [VPL 60-1032] Extracting hardware platform to
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/vivado/vpl/.local/hw_platform
[16:45:27] Run vpl: Step create_project: Started
Creating Vivado project.
[16:46:03] Run vpl: Step create_project: Completed
[16:46:03] Run vpl: Step create_bd: Started
[16:46:12] Run vpl: Step create_bd: Completed
[16:46:12] Run vpl: Step update_bd: Started

[16:46:15] Run vpl: Step update_bd: Completed

[16:46:15] Run vpl: Step generate_target: Started

[16:47:31] Run vpl: Step generate_target: RUNNING...

[16:48:46] Run vpl: Step generate_target: RUNNING...

[16:49:42] Run vpl: Step generate_target: Completed

[16:49:42] Run vpl: Step config_hw_emu.gen_scripts: Started

[16:50:27] Run vpl: Step config_hw_emu.gen_scripts: Completed

[16:50:27] Run vpl: Step config_hw_emu.compile: Started

[16:50:42] Run vpl: Step config_hw_emu.compile: Completed

[16:50:42] Run vpl: Step config_hw_emu.elaborate: Started

WARNING: [IP-LOCK-01] In your vivado project:prj for design:dr the IP pfm_top_axi_vip_0_0 is locked. The IP is locked for the following reason - * IP definition 'AXI Verification IP (1.1)' for IP 'pfm_top_axi_vip_0_0' (customized with software release 2022.1.1) has a different revision in the IP Catalog.

.
WARNING: [IP-LOCK-01] In your vivado project:prj for design:dr the IP pfm_top_connect_to_es_cu_0 is locked. The IP is locked for the following reason - * IP definition 'AXI Interconnect (2.1)' for IP 'pfm_top_connect_to_es_cu_0' (customized with software release 2022.1.1) has a different revision in the IP Catalog.

.
WARNING: [IP-LOCK-01] In your vivado project:prj for design:dr the IP pfm_top_smartconnect_0_0 is locked. The IP is locked for the following reason - * IP definition 'AXI SmartConnect (1.0)' for IP 'pfm_top_smartconnect_0_0' (customized with software release 2022.1.1) has a different revision in the IP Catalog.

* IP 'pfm_top_smartconnect_0_0' contains one or more locked subcores.* Target IP definition 'AXI SmartConnect (1.0)' requires a revision change. Please review the change log before upgrading the IP.

.
WARNING: [IP-LOCK-01] In your vivado project:prj for design:dr the IP pfm_top_xbar_0 is locked. The IP is locked for the following reason - * IP definition 'AXI Crossbar (2.1)' for IP 'pfm_top_xbar_0' (customized with software release 2022.1.1) has a different revision in the IP Catalog.

.
WARNING: [IP-LOCK-01] In your vivado project:prj for design:dr the IP bd_387c_one_0 is locked. The IP is locked for the following reason - * The IP Data in the repository is incompatible with the current instance (despite having identical Version and Revision). You will need to upgrade the IP before viewing the customization and generating outputs.

.
WARNING: [IP-LOCK-01] In your vivado project:prj for design:dr the IP bd_387c_s00mmu_0 is locked. The IP is locked for the following reason - * IP definition 'SC MMU (1.0)' for IP 'bd_387c_s00mmu_0' (customized with software release 2022.1.1) has a different revision in the IP Catalog.

.

WARNING: [IP-LOCK-01] In your vivado project:prj for design:dr the IP bd_387c_s00sic_0 is locked. The IP is locked for the following reason - * IP definition 'SC SI_CONVERTER (1.0)' for IP 'bd_387c_s00sic_0' (customized with software release 2022.1.1) has a different revision in the IP Catalog.

Check VPL, containing 2 checks, has run: 0 errors, 7 warning violations

[16:51:18] Run vpl: Step config_hw_emu.elaborate: Completed

[16:51:19] Run vpl: FINISHED. Run Status: config_hw_emu.elaborate Complete!

INFO: [v++ 60-1441] [16:51:19] Run run_link: Step vpl: Completed

Time (s): cpu = 00:00:09 ; elapsed = 00:06:08 . Memory (MB): peak = 436.348 ; gain = 0.000 ; free physical = 294426 ; free virtual = 455444

INFO: [v++ 60-1443] [16:51:19] Run run_link: Step rtdgen: Started

INFO: [v++ 60-1453] Command Line: rtdgen

INFO: [v++ 60-1454] Run Directory:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link

INFO: [v++ 60-991] clock name 'kernel_clk/clk' (clock ID '0') is being mapped to clock name 'DATA_CLK' in the xclbin

INFO: [v++ 60-1230] The compiler selected the following frequencies for the runtime controllable kernel clock(s) and scalable system clock(s): Kernel (DATA) clock: kernel_clk/clk = 300, Kernel (KERNEL) clock: kernel_clk/clk = 300

INFO: [v++ 60-1453] Command Line: cf2sw -a

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/address_map.xml -sdsl

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/sdsl.dat -xclbin

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/xclbin_orig.xml -rtd

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.rtd -o

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml

INFO: [v++ 60-1652] Cf2sw returned exit code: 0

WARNING: [v++ 60-1455] Debuggable symbols are not generated successfully, clean up

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/consolidated.cf

INFO: [v++ 60-1441] [16:51:27] Run run_link: Step rtdgen: Completed

Time (s): cpu = 00:00:08 ; elapsed = 00:00:08 . Memory (MB): peak = 436.348 ; gain = 0.000 ; free physical = 299659 ; free virtual = 460710

INFO: [v++ 60-1443] [16:51:27] Run run_link: Step xclbinutil: Started

INFO: [v++ 60-1453] Command Line: xclbinutil --add-section

BITSTREAM:RAW:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/behav.xse --force --target hw_emu --key-value

SYS:dfx_enable:false --add-section

```
:JSON:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.rtd --add-section
CLOCK_FREQ_TOPOLOGY:JSON:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml.rtd --add-section
BUILD_METADATA:JSON:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link_build.rtd --add-section
EMBEDDED_METADATA:RAW:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml --add-section
SYSTEM_METADATA:RAW:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/systemDiagramModelSlrBaseAddress.json --key-value SYS:PlatformVBNV:xilinx_u280_gen3x16_xdma_1_202211_1 --output
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/.build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link
XRT Build Version: 2.14.354 (2022.2)
    Build Date: 2022-10-08 09:49:58
    Hash ID: 43926231f7183688add2dccfd391b36a1f000bea
Creating a default 'in-memory' xclbin image.
```

Section: 'BITSTREAM'(0) was successfully added.

Size : 48407853 bytes

Format : RAW

File :

'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/behav.xse'

Section: 'MEM_TOPOLOGY'(6) was successfully added.

Format : JSON

File : 'mem_topology'

Section: 'IP_LAYOUT'(8) was successfully added.

Format : JSON

File : 'ip_layout'

Section: 'CONNECTIVITY'(7) was successfully added.

Format : JSON

File : 'connectivity'

Section: 'CLOCK_FREQ_TOPOLOGY'(11) was successfully added.

Size : 274 bytes

Format : JSON

File :
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml.rtd'

Section: 'BUILD_METADATA'(14) was successfully added.

Size : 2678 bytes

Format : JSON

File :
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link_build.rtd'

Section: 'EMBEDDED_METADATA'(2) was successfully added.

Size : 12216 bytes

Format : RAW

File :
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml'

Section: 'SYSTEM_METADATA'(22) was successfully added.

Size : 21974 bytes

Format : RAW

File :
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/systemDiagramModelSlrBaseAddress.json'

Successfully wrote (48463756 bytes) to the output file:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin

Leaving xclbinutil.

INFO: [v++ 60-1441] [16:51:28] Run run_link: Step xclbinutil: Completed

Time (s): cpu = 00:00:00.19 ; elapsed = 00:00:00.64 . Memory (MB): peak = 436.348 ; gain = 0.000 ; free physical = 299620 ; free virtual = 460675

INFO: [v++ 60-1443] [16:51:28] Run run_link: Step xclbinutilinfo: Started

INFO: [v++ 60-1453] Command Line: xclbinutil --quiet --force --info

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin.info --input

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin

INFO: [v++ 60-1454] Run Directory:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link

INFO: [v++ 60-1441] [16:51:29] Run run_link: Step xclbinutilinfo: Completed

Time (s): cpu = 00:00:00.57 ; elapsed = 00:00:00.71 . Memory (MB): peak = 436.348 ; gain = 0.000 ; free physical = 300176 ; free virtual = 461231

INFO: [v++ 60-1443] [16:51:29] Run run_link: Step generate_sc_driver: Started

INFO: [v++ 60-1453] Command Line:
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link
INFO: [v++ 60-1441] [16:51:29] Run run_link: Step generate_sc_driver: Completed
Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 436.348 ; gain = 0.000 ;
free physical = 300176 ; free virtual = 461231
Check POST-VPL, containing 1 checks, has run: 0 errors
INFO: [v++ 60-244] Generating system estimate report...
INFO: [v++ 60-1092] Generated system estimate report:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link/system_estimate_vadd.link.txt
INFO: [v++ 60-586] Created
./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin
INFO: [v++ 60-1307] Run completed. Additional information can be found in:
Guidance:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link/v++_link_vadd.link_guidance.html
Steps Log File:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/logs/link/link.steps.log

INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the relevant reports.
Run the following command.
vitis_analyzer
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin.link_summary
INFO: [v++ 60-791] Total elapsed time: 0h 6m 53s
INFO: [v++ 60-1653] Closing dispatch client.
v++ -p ./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin -g
--save-temps -t hw_emu --platform
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm --package.out_dir ./package.hw_emu -o
./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin
Option Map File Used: '/tools/Xilinx/Vitis/2022.2/data/vitis/vpp/optMap.xml'

***** v++ v2022.2 (64-bit)
**** SW Build 3671529 on 2022-10-13-17:52:11
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.

INFO: [v++ 60-1306] Additional information associated with this v++ package can be found at:
Reports: /tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x/reports/package
Log files: /tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x/logs/package
Running Dispatch Server on port: 38675

INFO: [v++ 60-1548] Creating build summary session with primary output
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary, at Sun Apr 26 16:51:41 2026

INFO: [v++ 60-1315] Creating rulecheck session with output
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x/reports/package/v++_package_vadd_guidance.html', at Sun Apr 26 16:51:41 2026

INFO: [v++ 60-895] Target platform:
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm

INFO: [v++ 60-1578] This platform contains Xilinx Shell Archive
'/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw/hw.xsa'

INFO: [v++ 74-78] Compiler Version string: 2022.2

INFO: [v++ 60-2256] Packaging for hardware emulation

INFO: [v++ 60-2460] Successfully copied a temporary xclbin to the output xclbin:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin

INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the relevant reports.
Run the following command.

vitis_analyzer

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary

INFO: [v++ 60-791] Total elapsed time: 0h 0m 15s

INFO: [v++ 60-1653] Closing dispatch client.

=== HW_EMU RUN ===

```
cp -rf ./_x.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/emconfig.json .
```

```
XCL_EMULATION_MODE=hw_emu ./hello_world_xrt -x
```

```
./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin
```

Open the device0

Load the xclbin ./build_dir.hw_emu.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin

INFO: [HW-EMU 05] Path of the simulation directory :

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/.run/3806750/hw_em/device0/binary_0/behav_waveform/xsim

server socket name is /tmp/rsotto/device0_0_3806750

INFO: [HW-EMU 01] Hardware emulation runs simulation underneath. Using a large data set will result in long simulation times. It is recommended that a small dataset is used for faster execution. The flow uses approximate models for Global memories and interconnect and hence the performance data generated is approximate.

configuring dataflow mode with ert polling

scheduler config ert(1), dataflow(1), slots(16), cudma(0), cuisr(0), cdma(0), cus(1)

Allocate Buffer in Global Memory

synchronize input buffer data to device global memory

Execution of the kernel

Get the output data from the device

TEST PASSED

INFO: [HW-EMU 06-0] Waiting for the simulator process to exit

INFO: [HW-EMU 06-1] All the simulator processes exited successfully

INFO: [HW-EMU 07-0] Please refer the path

"/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/.run/3806750/hw_em/device0/binary_0/behav_waveform/xsim/simulate.log" for more detailed simulation infos, errors and warnings.

=== HW BUILD ===

```
mkdir -p ./_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1
v++ -c --save-temps -t hw --platform
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm -k vadd --temp_dir ./_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1 -l'src'
-o'_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo' 'src/vadd.cpp'
Option Map File Used: '/tools/Xilinx/Vitis/2022.2/data/vitis/vpp/optMap.xml'
```

```
***** v++ v2022.2 (64-bit)
**** SW Build 3671529 on 2022-10-13-17:52:11
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
```

INFO: [v++ 60-1306] Additional information associated with this v++ compile can be found at:

Reports:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/reports/vadd

Log files:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/logs/vadd

Running Dispatch Server on port: 44507

INFO: [v++ 60-1548] Creating build summary session with primary output

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo.compile_summary, at Sun Apr 26 16:52:31 2026

INFO: [v++ 60-1315] Creating rulecheck session with output

'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/reports/vadd/v++_compile_vadd_guidance.html', at Sun Apr 26 16:52:31 2026

INFO: [v++ 60-895] Target platform:

/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm

INFO: [v++ 60-1578] This platform contains Xilinx Shell Archive

'/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw/hw.xsa'

INFO: [v++ 74-78] Compiler Version string: 2022.2

INFO: [v++ 60-585] Compiling for hardware target

INFO: [v++ 60-423] Target device: xilinx_u280_gen3x16_xdma_1_202211_1

INFO: [v++ 60-242] Creating kernel: 'vadd'

==>The following messages were generated while performing high-level synthesis for kernel:
vadd Log file:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd/vadd/vitis_hls.log :

INFO: [v++ 204-61] Pipelining loop 'mem_rd'.

INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 3, loop 'mem_rd'

INFO: [v++ 204-61] Pipelining loop 'mem_rd'.

INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 3, loop 'mem_rd'
INFO: [v++ 204-61] Pipelining loop 'mem_rd'.
INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 3, loop 'mem_rd'
INFO: [v++ 204-61] Pipelining loop 'execute'.
INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 3, Depth = 9, loop 'execute'
INFO: [v++ 204-61] Pipelining loop 'execute'.
INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 3, Depth = 9, loop 'execute'
INFO: [v++ 204-61] Pipelining loop 'mem_wr'.
INFO: [v++ 200-1470] Pipelining result : Target II = NA, Final II = 1, Depth = 3, loop 'mem_wr'
INFO: [v++ 200-789] **** Estimated Fmax: 411.00 MHz
INFO: [v++ 60-594] Finished kernel compilation
INFO: [v++ 60-244] Generating system estimate report...
INFO: [v++ 60-1092] Generated system estimate report:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/reports/vadd/system_estimate_vadd.txt
WARNING: [v++-17-1525]
INFO: [v++ 60-586] Created _x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo
INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the relevant reports.
Run the following command.

```
vitis_analyzer
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo.compile_summary
INFO: [v++ 60-791] Total elapsed time: 0h 0m 58s
INFO: [v++ 60-1653] Closing dispatch client.
mkdir -p ./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1
v++ -l --save-temps -t hw --platform
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm --temp_dir ./_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1
-o'./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin'
_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo
Option Map File Used: '/tools/Xilinx/Vitis/2022.2/data/vitis/vpp/optMap.xml'
```

***** v++ v2022.2 (64-bit)

**** SW Build 3671529 on 2022-10-13-17:52:11

** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.

INFO: [v++ 60-1306] Additional information associated with this v++ link can be found at:

Reports:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link

Log files:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/logs/link

Running Dispatch Server on port: 42053

INFO: [v++ 60-1548] Creating build summary session with primary output
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin.link_summary, at Sun Apr 26 16:53:31 2026

INFO: [v++ 60-1315] Creating rulecheck session with output
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link/v++_link_vadd.link_guidance.html', at Sun Apr 26 16:53:31 2026

INFO: [v++ 60-895] Target platform:
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm

INFO: [v++ 60-1578] This platform contains Xilinx Shell Archive
'/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw/hw.xsa'

INFO: [v++ 74-78] Compiler Version string: 2022.2

INFO: [v++ 60-629] Linking for hardware target

INFO: [v++ 60-423] Target device: xilinx_u280_gen3x16_xdma_1_202211_1

INFO: [v++ 60-1332] Run 'run_link' status: Not started

INFO: [v++ 60-1443] [16:53:35] Run run_link: Step system_link: Started

INFO: [v++ 60-1453] Command Line: system_link --xo
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo -keep --xpfm
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm --target hw --output_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int --temp_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link

INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link

INFO: [SYSTEM_LINK 82-70] Extracting xo v3 file
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xo

INFO: [SYSTEM_LINK 82-53] Creating IP database
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/_sys/.cdb/xd_ip_db.xml

INFO: [SYSTEM_LINK 82-38] [16:53:36] build_xd_ip_db started:
/tools/Xilinx/Vitis/2022.2/bin/build_xd_ip_db -ip_search 0 -sds-pf
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/hw.hpfm -clkid 0 -ip
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/iprepo/xilinx_com_hls_vadd_1_0,vadd -o
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/_sys/.cdb/xd_ip_db.xml

INFO: [SYSTEM_LINK 82-37] [16:53:41] build_xd_ip_db finished successfully

Time (s): cpu = 00:00:04 ; elapsed = 00:00:04 . Memory (MB): peak = 430.527 ; gain = 0.000 ; free physical = 307360 ; free virtual = 468166

INFO: [SYSTEM_LINK 82-51] Create system connectivity graph

INFO: [SYSTEM_LINK 82-102] Applying explicit connections to the system connectivity graph:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/cfgraph/cfggen_cfgraph.xml

INFO: [SYSTEM_LINK 82-38] [16:53:41] cfgen started: /tools/Xilinx/Vitis/2022.2/bin/cfgen -dpa_mem_offload false -dmclkid 0 -r
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/_sysl/.cdb/xd_ip_db.xml -o
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/cfgraph/cfggen_cfgraph.xml

INFO: [CFGGEN 83-0] Kernel Specs:

INFO: [CFGGEN 83-0] kernel: vadd, num: 1 {vadd_1}

INFO: [CFGGEN 83-2226] Inferring mapping for argument vadd_1.in1 to HBM[0]

INFO: [CFGGEN 83-2226] Inferring mapping for argument vadd_1.out to HBM[0]

INFO: [CFGGEN 83-2226] Inferring mapping for argument vadd_1.in2 to HBM[0]

INFO: [CFGGEN 83-2226] Inferring mapping for argument vadd_1.in3 to HBM[0]

INFO: [SYSTEM_LINK 82-37] [16:53:51] cfgen finished successfully

Time (s): cpu = 00:00:10 ; elapsed = 00:00:10 . Memory (MB): peak = 430.527 ; gain = 0.000 ; free physical = 303800 ; free virtual = 464608

INFO: [SYSTEM_LINK 82-52] Create top-level block diagram

INFO: [SYSTEM_LINK 82-38] [16:53:51] cf2bd started: /tools/Xilinx/Vitis/2022.2/bin/cf2bd --linux --trace_buffer 1024 --input_file
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/cfgraph/cfggen_cfgraph.xml --ip_db
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/_sysl/.cdb/xd_ip_db.xml --cf_name dr --working_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/_sysl/.xsd --temp_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link --output_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int --target_bd ulp.bd

INFO: [CF2BD 82-31] Launching cf2xd: cf2xd -linux -trace-buffer 1024 -i
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/cfgraph/cfggen_cfgraph.xml -r
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/_sysl/.cdb/xd_ip_db.xml -o dr.xml

INFO: [CF2BD 82-28] cf2xd finished successfully

INFO: [CF2BD 82-31] Launching cf_xsd: cf_xsd -disable-address-gen -bd ulp.bd -dn dr -dp
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/sys_link/_sysl/.xsd

INFO: [CF2BD 82-28] cf_xsd finished successfully

INFO: [SYSTEM_LINK 82-37] [16:53:55] cf2bd finished successfully
Time (s): cpu = 00:00:04 ; elapsed = 00:00:04 . Memory (MB): peak = 430.527 ; gain = 0.000 ;
free physical = 303575 ; free virtual = 464389
INFO: [v++ 60-1441] [16:53:55] Run run_link: Step system_link: Completed
Time (s): cpu = 00:00:20 ; elapsed = 00:00:20 . Memory (MB): peak = 436.348 ; gain = 0.000 ;
free physical = 303646 ; free virtual = 464461
INFO: [v++ 60-1443] [16:53:55] Run run_link: Step cf2sw: Started
INFO: [v++ 60-1453] Command Line: cf2sw -sdsl
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/sdsl.dat -rtd
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/cf2sw.rtd -nofilter
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/cf2sw_full.rtd -xclbin
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/xclbin_orig.xml -o
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/xclbin_orig.1.xml
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/run_link
INFO: [v++ 60-1441] [16:54:00] Run run_link: Step cf2sw: Completed
Time (s): cpu = 00:00:06 ; elapsed = 00:00:06 . Memory (MB): peak = 436.348 ; gain = 0.000 ;
free physical = 303337 ; free virtual = 464154
INFO: [v++ 60-1443] [16:54:00] Run run_link: Step rtd2_system_diagram: Started
INFO: [v++ 60-1453] Command Line: rtd2SystemDiagram
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/run_link
INFO: [v++ 60-1441] [16:54:01] Run run_link: Step rtd2_system_diagram: Completed
Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.29 . Memory (MB): peak = 436.348 ; gain = 0.000
; free physical = 303270 ; free virtual = 464080
INFO: [v++ 60-1443] [16:54:01] Run run_link: Step vpl: Started
INFO: [v++ 60-1453] Command Line: vpl -t hw -f
/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_2
02211_1.xpfm -s --remote_ip_cache
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/.ipcache --output_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int --log_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/logs/link --report_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/reports/link --config
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_

```
1_202211_1/link/int/vplConfig.ini -k
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/kernel_info.dat --webtalk_flag Vitis --temp_dir
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link --no-info --iprepo
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/xo/ip_repo/xilinx_com_hls_vadd_1_0 --messageDb
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/run_link/vpl.pb
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/dr.bd.tcl
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/run_link
```

```
***** vpl v2022.2 (64-bit)
**** SW Build 3671529 on 2022-10-13-17:52:11
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
```

```
INFO: [VPL 60-839] Read in kernel information from file
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/int/kernel_info.dat'.
INFO: [VPL 74-78] Compiler Version string: 2022.2
INFO: [VPL 60-423] Target device: xilinx_u280_gen3x16_xdma_1_202211_1
INFO: [VPL 60-1032] Extracting hardware platform to
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_
1_202211_1/link/vivado/vpl/.local/hw_platform
[16:54:15] Run vpl: Step create_project: Started
Creating Vivado project.
[16:54:21] Run vpl: Step create_project: Completed
[16:54:21] Run vpl: Step create_bd: Started
[16:54:52] Run vpl: Step create_bd: Completed
[16:54:52] Run vpl: Step update_bd: Started
[16:54:52] Run vpl: Step update_bd: Completed
[16:54:52] Run vpl: Step generate_target: Started
[16:55:48] Run vpl: Step generate_target: Completed
[16:55:48] Run vpl: Step config_hw_runs: Started
[16:55:51] Run vpl: Step config_hw_runs: Completed
[16:55:51] Run vpl: Step synth: Started
[16:56:22] Block-level synthesis in progress, 0 of 7 jobs complete, 6 jobs running.
[16:56:52] Block-level synthesis in progress, 0 of 7 jobs complete, 6 jobs running.
[16:57:22] Block-level synthesis in progress, 2 of 7 jobs complete, 4 jobs running.
[16:57:53] Block-level synthesis in progress, 4 of 7 jobs complete, 2 jobs running.
[16:58:23] Block-level synthesis in progress, 6 of 7 jobs complete, 1 job running.
```

[16:58:53] Block-level synthesis in progress, 6 of 7 jobs complete, 1 job running.
[16:59:23] Block-level synthesis in progress, 7 of 7 jobs complete, 0 jobs running.
[16:59:53] Top-level synthesis in progress.
[17:00:23] Top-level synthesis in progress.
[17:00:53] Top-level synthesis in progress.
[17:01:07] Run vpl: Step synth: Completed
[17:01:07] Run vpl: Step impl: Started
[17:09:40] Finished 2nd of 6 tasks (FPGA linking synthesized kernels to platform). Elapsed time:
00h 15m 37s

[17:09:40] Starting logic optimization..
[17:10:40] Phase 1 Retarget
[17:11:10] Phase 2 Constant propagation
[17:11:10] Phase 3 Sweep
[17:11:41] Phase 4 BUFG optimization
[17:12:11] Phase 5 Shift Register Optimization
[17:12:11] Phase 6 Post Processing Netlist
[17:13:11] Finished 3rd of 6 tasks (FPGA logic optimization). Elapsed time: 00h 03m 31s

[17:13:11] Starting logic placement..
[17:13:41] Phase 1 Placer Initialization
[17:13:41] Phase 1.1 Placer Initialization Netlist Sorting
[17:17:12] Phase 1.2 IO Placement/ Clock Placement/ Build Placer Device
[17:17:43] Phase 1.3 Build Placer Netlist Model
[17:19:43] Phase 1.4 Constrain Clocks/Macros
[17:20:13] Phase 2 Global Placement
[17:20:13] Phase 2.1 Floorplanning
[17:20:44] Phase 2.1.1 Partition Driven Placement
[17:20:44] Phase 2.1.1.1 PBP: Partition Driven Placement
[17:20:44] Phase 2.1.1.2 PBP: Clock Region Placement
[17:21:14] Phase 2.1.1.3 PBP: Compute Congestion
[17:21:44] Phase 2.1.1.4 PBP: UpdateTiming
[17:21:44] Phase 2.1.1.5 PBP: Add part constraints
[17:21:44] Phase 2.2 Physical Synthesis After Floorplan
[17:22:14] Phase 2.3 Update Timing before SLR Path Opt
[17:22:14] Phase 2.4 Post-Processing in Floorplanning
[17:22:14] Phase 2.5 Global Placement Core
[17:32:19] Phase 2.5.1 UpdateTiming Before Physical Synthesis
[17:32:49] Phase 2.5.2 Physical Synthesis In Placer
[17:33:49] Phase 3 Detail Placement
[17:34:20] Phase 3.1 Commit Multi Column Macros
[17:34:20] Phase 3.2 Commit Most Macros & LUTRAMs
[17:34:50] Phase 3.3 Small Shape DP
[17:34:50] Phase 3.3.1 Small Shape Clustering

[17:35:20] Phase 3.3.2 Flow Legalize Slice Clusters
[17:35:20] Phase 3.3.3 Slice Area Swap
[17:35:50] Phase 3.3.3.1 Slice Area Swap Initial
[17:36:21] Phase 3.4 Place Remaining
[17:36:21] Phase 3.5 Re-assign LUT pins
[17:36:21] Phase 3.6 Pipeline Register Optimization
[17:36:21] Phase 3.7 Fast Optimization
[17:36:51] Phase 4 Post Placement Optimization and Clean-Up
[17:36:51] Phase 4.1 Post Commit Optimization
[17:38:21] Phase 4.1.1 Post Placement Optimization
[17:38:21] Phase 4.1.1.1 BUFG Insertion
[17:38:21] Phase 1 Physical Synthesis Initialization
[17:38:21] Phase 4.1.1.2 BUFG Replication
[17:38:52] Phase 4.1.1.3 Post Placement Timing Optimization
[17:39:53] Phase 4.1.1.4 Replication
[17:40:53] Phase 4.2 Post Placement Cleanup
[17:40:53] Phase 4.3 Placer Reporting
[17:40:53] Phase 4.3.1 Print Estimated Congestion
[17:40:53] Phase 4.4 Final Placement Cleanup
[17:43:24] Finished 4th of 6 tasks (FPGA logic placement). Elapsed time: 00h 30m 12s

[17:43:24] Starting logic routing..
[17:43:54] Phase 1 Build RT Design
[17:45:25] Phase 2 Router Initialization
[17:45:25] Phase 2.1 Fix Topology Constraints
[17:45:25] Phase 2.2 Pre Route Cleanup
[17:45:55] Phase 2.3 Global Clock Net Routing
[17:46:25] Phase 2.4 Update Timing
[17:47:26] Phase 2.5 Update Timing for Bus Skew
[17:47:26] Phase 2.5.1 Update Timing
[17:47:56] Phase 3 Initial Routing
[17:47:56] Phase 3.1 Global Routing
[17:48:26] Phase 4 Rip-up And Reroute
[17:48:26] Phase 4.1 Global Iteration 0
[17:53:28] Phase 4.2 Global Iteration 1
[17:54:59] Phase 4.3 Global Iteration 2
[17:55:29] Phase 5 Delay and Skew Optimization
[17:55:29] Phase 5.1 Delay CleanUp
[17:55:29] Phase 5.1.1 Update Timing
[17:56:00] Phase 5.1.2 Update Timing
[17:56:30] Phase 5.2 Clock Skew Optimization
[17:56:30] Phase 6 Post Hold Fix
[17:56:30] Phase 6.1 Hold Fix Iter
[17:56:30] Phase 6.1.1 Update Timing

[17:57:00] Phase 7 Leaf Clock Prog Delay Opt
[17:57:30] Phase 8 Route finalize
[17:57:30] Phase 9 Verifying routed nets
[17:58:00] Phase 10 Depositing Routes
[17:58:31] Phase 11 Resolve XTalk
[17:58:31] Phase 12 Post Router Timing
[17:58:31] Finished 5th of 6 tasks (FPGA routing). Elapsed time: 00h 15m 06s

[17:58:31] Starting bitstream generation..

[18:06:34] Creating bitmap...

[18:17:09] Writing bitstream ./level0_i_ulp_my_rm_partial.bit...

Check VPL, containing 1 checks, has run: 0 errors

[18:17:24] Run vpl: Step impl: Completed

[18:17:25] Finished 6th of 6 tasks (FPGA bitstream generation). Elapsed time: 00h 18m 53s

[18:17:25] Run vpl: FINISHED. Run Status: impl Complete!

INFO: [v++ 60-1441] [18:17:25] Run run_link: Step vpl: Completed

Time (s): cpu = 00:00:31 ; elapsed = 01:23:24 . Memory (MB): peak = 436.348 ; gain = 0.000 ;
free physical = 288348 ; free virtual = 455396

INFO: [v++ 60-1443] [18:17:25] Run run_link: Step rtdgen: Started

INFO: [v++ 60-1453] Command Line: rtdgen

INFO: [v++ 60-1454] Run Directory:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link

INFO: [v++ 60-991] clock name 'ulp_ucs/aclk_kernel_01' (clock ID '1') is being mapped to clock name 'KERNEL_CLK' in the xclbin

INFO: [v++ 60-991] clock name 'ulp_ucs/aclk_kernel_00' (clock ID '0') is being mapped to clock name 'DATA_CLK' in the xclbin

INFO: [v++ 60-991] clock name 'hbm_aclk' (clock ID '') is being mapped to clock name 'hbm_aclk' in the xclbin

INFO: [v++ 60-1230] The compiler selected the following frequencies for the runtime controllable kernel clock(s) and scalable system clock(s): System (SYSTEM) clock: hbm_aclk = 450, Kernel (KERNEL) clock: ulp_ucs/aclk_kernel_01 = 500, Kernel (DATA) clock: ulp_ucs/aclk_kernel_00 = 300

INFO: [v++ 60-1453] Command Line: cf2sw -a

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/address_map.xml -sdsl

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/sdsl.dat -xclbin

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/xclbin_orig.xml -rtd

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.rtd -o

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml

INFO: [v++ 60-1652] Cf2sw returned exit code: 0
INFO: [v++ 60-1441] [18:17:30] Run run_link: Step rtdgen: Completed
Time (s): cpu = 00:00:05 ; elapsed = 00:00:05 . Memory (MB): peak = 436.348 ; gain = 0.000 ;
free physical = 289992 ; free virtual = 457051
INFO: [v++ 60-1443] [18:17:30] Run run_link: Step xclbinutil: Started
INFO: [v++ 60-1453] Command Line: xclbinutil --add-section
BITSTREAM:RAW:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/partial.bit --force --target hw --key-value
SYS:dfx_enable:true --add-section
:JSON:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.rtd --append-section
:JSON:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/appendSection.rtd --add-section
CLOCK_FREQ_TOPOLOGY:JSON:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml.rtd --add-section
BUILD_METADATA:JSON:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link_build.rtd --add-section
EMBEDDED_METADATA:RAW:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml --add-section
SYSTEM_METADATA:RAW:/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/systemDiagramModelSlrBaseAddress.json
--key-value SYS:PlatformVBNV:xilinx_u280_gen3x16_xdma_1_202211_1 --output
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/.build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin
INFO: [v++ 60-1454] Run Directory:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link
XRT Build Version: 2.14.354 (2022.2)
Build Date: 2022-10-08 09:49:58
Hash ID: 43926231f7183688add2dccfd391b36a1f000bea
Creating a default 'in-memory' xclbin image.

Section: 'BITSTREAM'(0) was successfully added.

Size : 50224718 bytes

Format : RAW

File :

'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/partial.bit'

Section: 'MEM_TOPOLOGY'(6) was successfully added.

Format : JSON

File : 'mem_topology'

Section: 'IP_LAYOUT'(8) was successfully added.

Format : JSON
File : 'ip_layout'

Section: 'CONNECTIVITY'(7) was successfully added.

Format : JSON
File : 'connectivity'

Section: 'CLOCK_FREQ_TOPOLOGY'(11) was successfully added.

Size : 410 bytes
Format : JSON
File :
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml.rtd'

Section: 'BUILD_METADATA'(14) was successfully added.

Size : 2623 bytes
Format : JSON
File :
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link_build.rtd'

Section: 'EMBEDDED_METADATA'(2) was successfully added.

Size : 12257 bytes
Format : RAW
File :
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/vadd.link.xml'

Section: 'SYSTEM_METADATA'(22) was successfully added.

Size : 26687 bytes
Format : RAW
File :
'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/int/systemDiagramModelSlrBaseAddress.json'

Section: 'PARTITION_METADATA'(20) was successfully appended to.

Format : JSON
File : 'partition_metadata'

Section: 'IP_LAYOUT'(8) was successfully appended to.

Format : JSON
File : 'ip_layout'

Successfully wrote (50295376 bytes) to the output file:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin

Leaving xclbinutil.

INFO: [v++ 60-1441] [18:17:30] Run run_link: Step xclbinutil: Completed

Time (s): cpu = 00:00:00.2 ; elapsed = 00:00:00.6 . Memory (MB): peak = 436.348 ; gain = 0.000 ; free physical = 289592 ; free virtual = 456653

INFO: [v++ 60-1443] [18:17:30] Run run_link: Step xclbinutilinfo: Started

INFO: [v++ 60-1453] Command Line: xclbinutil --quiet --force --info

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin.info --input

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin

INFO: [v++ 60-1454] Run Directory:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link

INFO: [v++ 60-1441] [18:17:31] Run run_link: Step xclbinutilinfo: Completed

Time (s): cpu = 00:00:00.61 ; elapsed = 00:00:00.72 . Memory (MB): peak = 436.348 ; gain = 0.000 ; free physical = 289064 ; free virtual = 456127

INFO: [v++ 60-1443] [18:17:31] Run run_link: Step generate_sc_driver: Started

INFO: [v++ 60-1453] Command Line:

INFO: [v++ 60-1454] Run Directory:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/link/run_link

INFO: [v++ 60-1441] [18:17:31] Run run_link: Step generate_sc_driver: Completed

Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 436.348 ; gain = 0.000 ; free physical = 289063 ; free virtual = 456126

Check POST-VPL, containing 1 checks, has run: 0 errors

INFO: [v++ 60-244] Generating system estimate report...

INFO: [v++ 60-1092] Generated system estimate report:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link/system_estimate_vadd.link.txt

INFO: [v++ 60-586] Created

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.ltx

INFO: [v++ 60-586] Created

./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin

INFO: [v++ 60-1307] Run completed. Additional information can be found in:

Guidance:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link/v++_link_vadd.link_guidance.html

Timing Report:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/reports/link/imp/impl_1_hw_bb_locked_timing_summary_routed.rpt

Vivado Log:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/logs/link/vivado.log

Steps Log File:

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x.hw.xilinx_u280_gen3x16_xdma_1_202211_1/logs/link/link.steps.log

INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the relevant reports.
Run the following command.

vitis_analyzer

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin.link_summary

INFO: [v++ 60-791] Total elapsed time: 1h 24m 10s

INFO: [v++ 60-1653] Closing dispatch client.

v++ -p ./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.link.xclbin --save-temps -t hw --platform

/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm --package.out_dir ./package.hw -o

./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin

Option Map File Used: '/tools/Xilinx/Vitis/2022.2/data/vitis/vpp/optMap.xml'

***** v++ v2022.2 (64-bit)

**** SW Build 3671529 on 2022-10-13-17:52:11

** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.

INFO: [v++ 60-1306] Additional information associated with this v++ package can be found at:

Reports: /tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x/reports/package

Log files: /tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x/logs/package

Running Dispatch Server on port: 34573

INFO: [v++ 60-1548] Creating build summary session with primary output

/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary, at Sun Apr 26 18:17:53 2026

INFO: [v++ 60-1315] Creating rulecheck session with output

'/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/_x/reports/package/v++_package_vadd_guidance.html', at Sun Apr 26 18:17:53 2026

INFO: [v++ 60-895] Target platform:

/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/xilinx_u280_gen3x16_xdma_1_202211_1.xpfm

INFO: [v++ 60-1578] This platform contains Xilinx Shell Archive

'/opt/xilinx/platforms/xilinx_u280_gen3x16_xdma_1_202211_1/hw/hw.xsa'

INFO: [v++ 74-78] Compiler Version string: 2022.2

INFO: [v++ 60-2256] Packaging for hardware

INFO: [v++ 60-2460] Successfully copied a temporary xclbin to the output xclbin:
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin

INFO: [v++ 60-2343] Use the vitis_analyzer tool to visualize and navigate the relevant reports.
Run the following command.

 vitis_analyzer
/tools/Xilinx/home/rsotto/Vitis_Accel_Examples/hello_world/build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin.package_summary

INFO: [v++ 60-791] Total elapsed time: 0h 0m 14s

INFO: [v++ 60-1653] Closing dispatch client.

=== HW RUN ===

```
./hello_world_xrt -x ./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin
```

Open the device0

Load the xclbin ./build_dir.hw.xilinx_u280_gen3x16_xdma_1_202211_1/vadd.xclbin

Allocate Buffer in Global Memory

synchronize input buffer data to device global memory

Execution of the kernel

Get the output data from the device

TEST PASSED

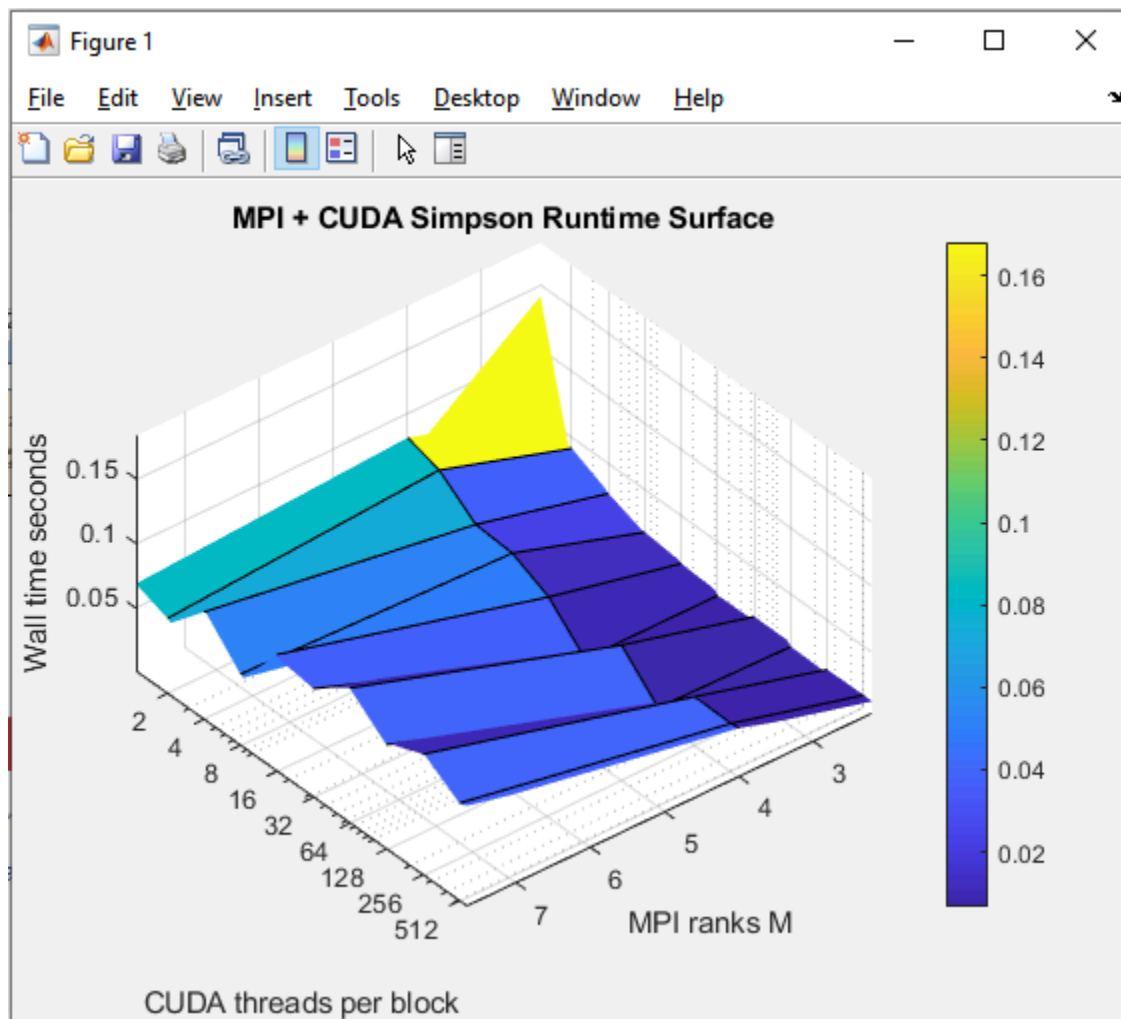
PA11

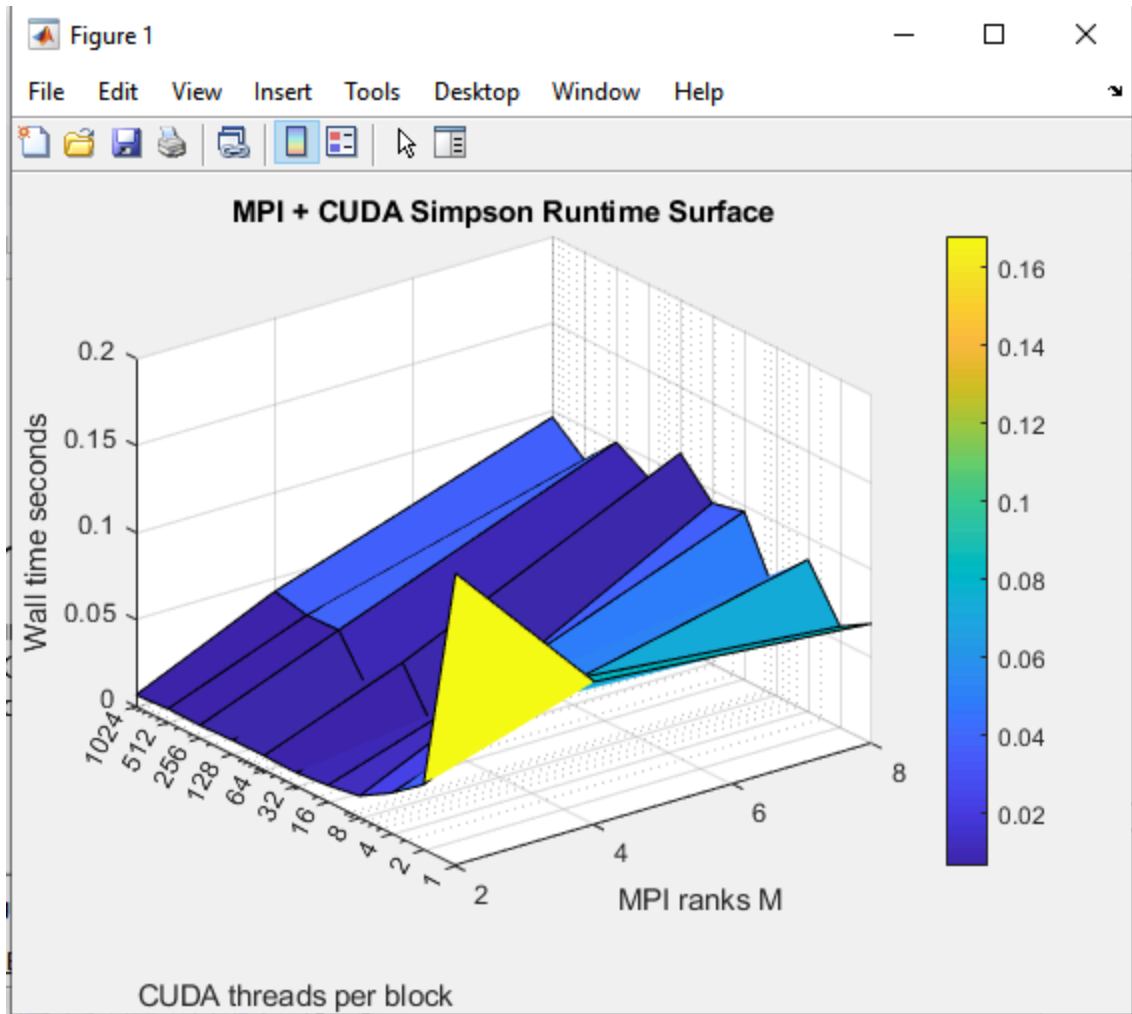
Comments

After running with $N = 100000000$, the optimal configuration I found occurred with MPI ranks = 2, and CUDA thread count = 1024. I encountered errors while trying to run with thread count = 2048 across all MPI rank counts.

Overall, increasing CUDA thread count was effective for small MPI rank count, but led to inconsistent performance with ranks higher than 2, shown as the multiple ridges in the surface plot. Increasing MPI rank count surprisingly led to a decrease in performance with CUDA thread counts past 2-4 threads per block. This result can likely be attributed to the overhead of using MPI, where message passing between processes outweighed the speedup gained from a more distributed workload.

MPI Ranks vs CUDA Thread Count vs Runtime Plot





MPI + CUDA Code – simpsons_mpi_cuda.cu

```
#include <mpi.h>
#include <cuda_runtime.h>

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>

#define CUDA_CHECK(call) do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error %s:%d: %s\n", \
            __FILE__, __LINE__, cudaGetErrorString(err)); \
        MPI_Abort(MPI_COMM_WORLD, 1); \
    } \
} while (0)

__device__ double f_gpu(double x) {
    double c = cos(x);
    return acos(c / (1.0 + 2.0 * c));
}

__global__ void simpson_kernel(
    unsigned long long j_start,
    unsigned long long j_end,
    double a,
    double h,
    double *block_sums
){
    extern __shared__ double shmem[];

    unsigned int tid = threadIdx.x;
    unsigned long long global_tid = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned long long stride = gridDim.x * blockDim.x;

    double sum = 0.0;

    for (unsigned long long j = j_start + global_tid; j <= j_end; j += stride) {
        sum += f_gpu(a + (2 * j - 2) * h)
            + 4.0 * f_gpu(a + (2 * j - 1) * h)
            + f_gpu(a + (2 * j) * h);
    }
}
```

```

shmem[tid] = sum;
__syncthreads();

for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
    if (tid < s) {
        shmem[tid] += shmem[tid + s];
    }
    __syncthreads();
}

if (tid == 0) {
    block_sums[blockIdx.x] = shmem[0];
}
}

unsigned long long read_ull_arg(int argc, char **argv, const char *flag, unsigned long long def) {
    for (int i = 1; i < argc - 1; i++) {
        if (strcmp(argv[i], flag) == 0) {
            return strtoull(argv[i + 1], NULL, 10);
        }
    }
    return def;
}

int read_int_arg(int argc, char **argv, const char *flag, int def) {
    for (int i = 1; i < argc - 1; i++) {
        if (strcmp(argv[i], flag) == 0) {
            return atoi(argv[i + 1]);
        }
    }
    return def;
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, M;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &M);

    unsigned long long N = read_ull_arg(argc, argv, "-n", 1000000ULL);
    int blockDim = read_int_arg(argc, argv, "-t", 256);

```

```

if (N % 2 != 0) {
    if (rank == 0) {
        fprintf(stderr, "Error: N must be even for composite Simpson's 1/3 rule.\n");
    }
    MPI_Finalize();
    return 1;
}

```

```

if ((blockDim & (blockDim - 1)) != 0) {
    if (rank == 0) {
        fprintf(stderr, "Error: CUDA thread count must be a power of 2.\n");
    }
    MPI_Finalize();
    return 1;
}

```

```

int device_count = 0;
CUDA_CHECK(cudaGetDeviceCount(&device_count));
CUDA_CHECK(cudaSetDevice(rank % device_count));

```

```

double a = 0.0;
double b = M_PI / 2.0;
double exact = 5.0 * M_PI * M_PI / 24.0;
double h = (b - a) / (double)N;

```

```

unsigned long long num_pairs = N / 2;

```

```

unsigned long long j_start = rank * num_pairs / M + 1;
unsigned long long j_end   = (rank + 1) * num_pairs / M;

```

```

unsigned long long local_pairs = 0;
if (j_end >= j_start) {
    local_pairs = j_end - j_start + 1;
}

```

```

int gridDim = 1024;
if (local_pairs < (unsigned long long)gridDim * blockDim) {
    gridDim = (int)((local_pairs + blockDim - 1) / blockDim);
    if (gridDim < 1) gridDim = 1;
}

```

```

double *d_block_sums = NULL;
double *h_block_sums = (double *)calloc(gridDim, sizeof(double));

```

```
CUDA_CHECK(cudaMalloc((void **)&d_block_sums, gridDim * sizeof(double)));
```

```
MPI_Barrier(MPI_COMM_WORLD);
```

```
double t0 = MPI_Wtime();
```

```
simpson_kernel<<<gridDim, blockDim, blockDim * sizeof(double)>>>(  
    j_start,  
    j_end,  
    a,  
    h,  
    d_block_sums  
);
```

```
CUDA_CHECK(cudaGetLastError());
```

```
CUDA_CHECK(cudaDeviceSynchronize());
```

```
CUDA_CHECK(cudaMemcpy(  
    h_block_sums,  
    d_block_sums,  
    gridDim * sizeof(double),  
    cudaMemcpyDeviceToHost  
));
```

```
double local_sum = 0.0;  
for (int i = 0; i < gridDim; i++) {  
    local_sum += h_block_sums[i];  
}
```

```
double local_approx = local_sum * h / 3.0;  
double approx = 0.0;
```

```
MPI_Reduce(  
    &local_approx,  
    &approx,  
    1,  
    MPI_DOUBLE,  
    MPI_SUM,  
    0,  
    MPI_COMM_WORLD  
);
```

```
double t1 = MPI_Wtime();  
double local_time = t1 - t0;  
double wall_time = 0.0;
```

```
MPI_Reduce(
    &local_time,
    &wall_time,
    1,
    MPI_DOUBLE,
    MPI_MAX,
    0,
    MPI_COMM_WORLD
);

if (rank == 0) {
    double error = fabs(exact - approx);
    printf("M=%d threads=%d N=%llu approx=%.15f exact=%.15f error=%.15e time=%.8f\n",
        M, blockDim, N, approx, exact, error, wall_time);
}

CUDA_CHECK(cudaFree(d_block_sums));
free(h_block_sums);

MPI_Finalize();
return 0;
}
```

Makefile

CXX = mpicxx

NVCC = nvcc

TARGET = simpson_mpi_cuda

SRC = simpson_mpi_cuda.cu

N ?= 100000000

OUT ?= results.txt

NVCCFLAGS = -O3 -std=c++11 -ccbin \$(CXX)

all: \$(TARGET)

\$(TARGET): \$(SRC)

\$(NVCC) \$(NVCCFLAGS) \$(SRC) -o \$(TARGET)

run: \$(TARGET)

@rm -f \$(OUT)

@echo "Running tests with N=\$(N)"

@for M in 2 4 8; do \

for T in 1 2 4 8 16 32 64 128 256 512 1024 2048; do \

echo "M=\$\$M T=\$\$T"; \

mpirun -mca btl "^openib" -np \$\$M ./\$(TARGET) -n \$(N) -t \$\$T >>

\$(OUT); \

done; \

done

@echo "Results saved to \$(OUT)"

clean:

rm -f \$(TARGET) *.o \$(OUT) runtime_surface.png

Matlab Code - plot_surface.m

```
function plot_surface(filename)
    if nargin < 1
        filename = 'results.txt';
    end
    fid = fopen(filename, 'r');
    if fid < 0
        error('Could not open %s', filename);
    end
    M_vals = [];
    T_vals = [];
    times = [];
    while ~feof(fid)
        line = fgetl(fid);
        tokens = regexp(line, ...
            'M=(\d+)\s+threads=(\d+)\.*time=([0-9.eE+-]+)', ...
            'tokens');
        if ~isempty(tokens)
            tok = tokens{1};
            M_vals(end+1) = str2double(tok{1});
            T_vals(end+1) = str2double(tok{2});
            times(end+1) = str2double(tok{3});
        end
    end
    fclose(fid);
    M_unique = unique(M_vals);
    T_unique = unique(T_vals);
    [X, Y] = meshgrid(M_unique, T_unique);
    Z = nan(size(X));
    for k = 1:length(times)
        i = find(T_unique == T_vals(k));
        j = find(M_unique == M_vals(k));
        Z(i, j) = times(k);
    end
    figure;
    surf(X, Y, Z);
    xlabel('MPI ranks M');
    ylabel('CUDA threads per block');
    zlabel('Wall time seconds');
    title('MPI + CUDA Simpson Runtime Surface');
    set(gca, 'YScale', 'log');
    yticks(T_unique);
    yticklabels(string(T_unique));
```

```
colorbar;  
grid on;  
saveas(gcf, 'runtime_surface.png');  
[best_time, idx] = min(times);  
fprintf('Best configuration:\n');  
fprintf('M=%d, threads=%d, time=%g seconds\n', ...  
        M_vals(idx), T_vals(idx), best_time);  
end
```